DOT-Design

Version 0.1.0, 2026-10-13

Overview

DOT-Design defines the portable model and reference tooling used to describe, validate, deploy, and compose Dynamic Open Trust (DOT) components in the MARE 6G security ecosystem.

A DOT packages four concerns without collapsing them into one runtime-specific description:

- **capability** — Skills describe what the component does;
- **contract** — OpenAPI, AsyncAPI, Protobuf, or GraphQL documents describe authoritative interactions;
- **composition** — Anchors give selected Contract operations stable identities and `PROVIDES` or `REQUIRES` roles; and
- **deployment** — Runtimes and deployment manifests describe where and how the component can run.



`dot.json` is the deployable DOT definition and the only source inspected by `dotx deploy`. Connections between deployed DOT instances are described in a separate composition document. Concrete addresses, credentials, and live policy results never become Anchor fields.

Documentation map

Read	When you need to
Getting started	Build the project, validate a DOT, inspect a composition plan, or generate this documentation.
Architecture	Understand repository boundaries, sources of truth, runtime/deployment separation, and the composition resolution path.
DOT Anchors and Composition	Author Anchors, bind them to services, connect DOT instances, materialize configuration, handle security inputs, and operate a composition.
Verification and examples	Choose conformance, dry-run, source-adapted, or live evidence and find the executable MARE PoC scenarios.
DOT-Domain	Work with the Java aggregate, value objects, validation, Contracts, Runtimes, and deployment manifests.
Integration handoff	Implement a catalogue, composer, or SFC adapter without coupling to a specific service, language, framework, or storage technology.

Repository components

Component	Responsibility	Build ecosystem
`DOT-Domain`	Java domain model and validation library. It is not a web application.	Maven / Java 26
`DOT-CLI`	Independent `dotx` CLI for authoring support, validation, inspection, deployment, runtime lifecycle, and the provisional composition executor.	Go 1.25+
`DOT-Schema`	Generated JSON Schema for `dot.json`, tracked directly in this repository.	JSON Schema
`DOT-Repository`	Git submodule containing reusable DOT definitions and samples.	Git submodule
`DOT-Skills`	Git submodule containing the Skills catalog.	Git submodule
`testing`	Repository-level conformance, composition, interoperability, and end-to-end evidence. It is test infrastructure, not a Maven module.	Maven, Go, shell, container tools

Supported Contract standards

Standard	Anchor selector
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
Protobuf	Fully qualified service method
GraphQL	Root-field schema coordinate

Contractless Anchors remain available for interactions that cannot be described by one of these standards. They must declare direction, protocol, and media type explicitly.

Maturity and scope

The DOT-owned Anchor model, service-port binding, generated schema, repository resolution, and conformance fixtures are release-candidate surfaces. The separate `dot.mare6g.eu/v1alpha1` composition document and `dotx compose` executor remain a reference implementation.

The reference executor is intentionally fail-closed. It does not claim to be a general multi-host orchestrator, trust-policy engine, active security scanner, or replacement for Kubernetes, Helm, Docker, Vagrant, or an SFC implementation. See the supported composition profile for the exact boundary.

First steps

```
# Domain unit tests; no MongoDB required
mvn -pl DOT-Domain test
```

```
# Install the pinned CLI checkout on PATH
task cli:install

# Validate and inspect the canonical composition example
dotx compose validate DOT-Testing/e2e/api-gateway-oidc/scenario.yaml
dotx compose plan DOT-Testing/e2e/api-gateway-oidc/scenario.yaml

# Run the complete Anchor/composition repository gate
task testing:conformance
```

Continue with Getting started for prerequisites and command context.

Project links

- MARE project
- MARE proof-of-concept catalogue
- DOT-Design source repository

Getting started

This page establishes a small, reproducible development path. Start with the domain, repository, and CLI checks before running live deployment scenarios.

Prerequisites

Tool	Required for	Project baseline
Java and Maven	`DOT-Domain`, schema generation, and Java tests	Java 26; Maven 3.9+
Go	Building and testing `dotx`	Go 1.25+
Task	Repository command aliases	Optional but recommended
Docker, Kubernetes tools, Helm, or Vagrant with libvirt/KVM	Only the matching deployment adapters and live scenarios	Install only what you intend to execute
Antora, the Antora PDF extension, Asciidoctor PDF, and Mermaid CLI	Building the HTML/PDF documentation site and Mermaid diagrams	See Build the documentation

Check out the repository

```
git clone https://github.com/MARE-EU-Project/DOT-Design.git
cd DOT-Design
git submodule update --init --recursive
```

The source repository and its submodules may require access granted by the MARE-EU-Project organization. `DOT-Schema` is tracked directly and does not require a separate checkout.

Verify the domain model

The domain module uses plain JUnit tests and does not start a service:

```
mvn -pl DOT-Domain test
```

Run the complete Java build with:

```
mvn clean install
# equivalent Task alias
task mci
```

The build generates CycloneDX JSON and XML software bills of materials during the Maven `verify` phase.

Install and test dotx

`DOT-CLI` is an independent Go module and is not part of the Maven reactor:

```
task cli:build
task cli:test
task cli:install
dotx --version
```

`task cli:install` installs the pinned checkout into `GOBIN` or `GOPATH/bin` and installs Bash completion. Keep that directory on `PATH`; project tasks, test harnesses, examples, and CI all invoke the same `dotx` command rather than a repository-local binary.

For a Vagrant runtime on Ubuntu, install or reconcile the complete provider boundary and then open a new login session so the `libvirt` and `kvm` groups are active:

```
dotx tool install vagrant
dotx tool check vagrant
```

The installer selects `qemu-kvm` when the distribution provides it and falls back to `qemu-system-x86` on releases such as Ubuntu 26.04. The readiness check also verifies the pinned `vagrant-libvirt` plugin, the `qemu:///system` connection, the default libvirt network, and `/dev/kvm` access.

Validate a DOT

`dotx validate` reads `dot.json` at the project root, falling back to `dot/dot.json`. The selected file is the unique DOT deployment definition; unreferenced Dockerfiles, Compose files, Kubernetes manifests, Helm charts, and scripts are not inferred.

```
dotx validate DOT-Repository/DOTs/testing/contract-standards/provider \
  --schema DOT-Schema/schema.json
dotx inspect DOT-Repository/DOTs/testing/contract-standards/provider --json
```

Validation checks the JSON Schema and domain relationships. Inspection provides a normalized, machine-readable view without deploying anything.

Validate and plan a composition

The canonical example connects Keycloak, APISIX, and an OAuth client:

```
dotx compose lock \
```

```
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml --check

dotx compose validate \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml

dotx compose plan \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml

dotx compose plan \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml --json

dotx compose apply \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml --dry-run
```

All five commands above are non-mutating. `lock --check` verifies the committed content-addressed input graph; `validate` answers whether references and compatibility can be resolved; `plan` shows the exact instances, endpoints, security status, and materializers; `apply --dry-run` renders supported runtime actions without writing composition state.

Read Validation, planning, and lifecycle before a live apply.

Verify the authoritative repository

Validate every admitted definition and inspect one pinned entry:

```
dotx repository test --repository DOT-Repository --schema DOT-Schema/schema.json
dotx repository inspect \
  d4a70000-0000-4000-8000-000000000001@1.0.0 \
  --repository DOT-Repository --json
```

Run the composition conformance gate

```
task testing:conformance
```

This gate combines Java domain tests, Go tests, repository-wide schema validation, Contract checks, composition validation/planning, and dry-run scenario checks. It does not claim that external partner systems or production networks were exercised.

Run only the versioned composition production profile with:

```
task testing:compose:acceptance
```

This gate writes revision-pinned, secret-free JSON evidence below `DOT-Testing/e2e/target/`.

Build the documentation

Install the documentation toolchain once:

```
task docs:setup
```

Then generate the Antora HTML site and PDF export:

```
task docs:check
task docs:build
```

Generated artifacts are written under `DOT-Docs/build/site` and are intentionally ignored by Git. To build and serve the site locally:

```
task docs:run
```

Open `http://localhost:8000`. Do not edit files under `DOT-Docs/build`; update the AsciiDoc sources instead.

Mermaid sources are stored as Antora attachments and rendered locally by `task docs:diagrams`. The renderer produces SVG for the HTML site and high-resolution PNG for PDF export; both outputs are committed so a clean checkout can resolve every Antora resource. Each source includes an accessible title and description, and each AsciiDoc image macro supplies alternative text.

`task docs:build` regenerates the diagrams before building either output. The toolchain is version-pinned and does not send architecture content to an external rendering service. Edit the `.mmd` source, never the generated SVG or PNG, then run the full documentation build to verify both renderers.

Where to continue

- Architecture for system and ownership boundaries.
- DOT Anchors and Composition for the model overview and task-oriented composition guides.
- Verification and examples for evidence levels and MARE PoC scenarios.
- DOT-Domain for Java model details.
- Integration handoff for catalogue and composer implementers.

Architecture

DOT-Design separates reusable component description from deployment-instance topology. That separation allows a DOT to be authored and validated once, then instantiated in multiple compositions without embedding addresses, credentials, or orchestrator-specific connection objects in the DOT aggregate.

The diagrams use a consistent visual language: blue identifies authored or persistent artifacts, teal identifies validation and resolution activities, amber identifies decisions or incomplete evidence, green identifies successful outcomes, and red identifies rejection. Labels and line styles carry the same meaning when the documentation is printed without color; dashed lines indicate optional, indirect, or evidence-only relationships.

System context

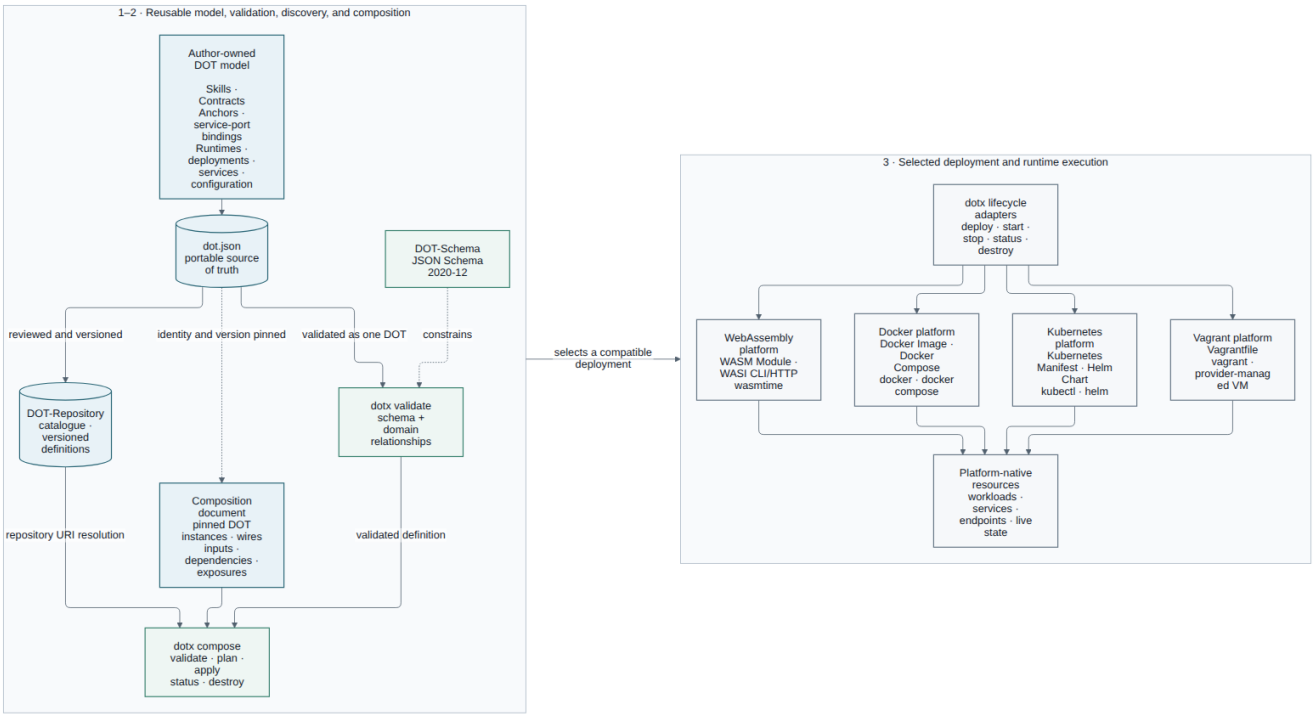


Figure 1. DOT-Design system context and runtime support

`DOT-Repository` is the authoritative catalogue of reusable definitions. `dotx` is the reference validation, resolution, deployment, and composition client. Platform-native systems remain responsible for workload execution and live state.

The runtime portion deliberately distinguishes **where** a workload runs from **how** it is deployed. The domain model supports four `PlatformEngine` values and six compatible manifest mappings:

- Docker accepts `DOCKER_IMAGE` and `DOCKER_COMPOSE`;
- Kubernetes accepts `KUBERNETES_MANIFEST` and `HELM_CHART`;
- Vagrant accepts `VAGRANTFILE`; and
- WebAssembly accepts `WASM_MODULE`.

The current dotx lifecycle implements Docker Image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and WebAssembly execution. The WebAssembly adapter uses Wasmtime with an explicit WASI_CLI or WASI_HTTP world, verifies the module's SHA-256 before execution, and grants only declared guest environment and directory capabilities.

Source-of-truth boundaries

Concern	Authoritative source	Explicitly excluded
DOT identity and supported capability	dot.json: id, name, version, Skills, and Contracts	Composition-local instance names and live state
Contract semantics	OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract payload	Repeated protocol, media, schema, or security fields on a contract-backed Anchor
Logical composition point	Dot.anchors[]	Hosts, URLs, credentials, placement, health, and policy decisions
Deployable endpoint declaration	DeploymentService.ports[] and anchorBindings[]	Allocated addresses, forwarded ports, and certificates
Runtime configuration surface	DeploymentService.config	Operator-supplied values already applied to one deployment
Multi-DOT topology and readiness	Composition document wires and dependsOn	Changes to either participating dot.json
Runtime state	Platform-native resources plus private dotx composition state	Contract or Anchor metadata

Repository architecture

```
DOT-Design
 ├─── DOT-Domain       Java model and validation
 ├─── DOT-Schema       generated dot.json schema
 ├─── DOT-CLI          independent Go CLI and reference executor
 ├─── DOT-Repository   reusable DOT definitions (submodule)
 ├─── DOT-Skills       Skills catalogue (submodule)
 ├─── testing          conformance, composition, interoperability, and E2E evidence
 └─── DOT-Docs         Antora sources, UI, playbooks, and generated diagrams
```

The Maven build contains the DOT-Domain Java module. DOT-CLI has its own Go module and lifecycle. testing is deliberately outside both module hierarchies. DOT-Repository is the only admitted source for valid deployable DOT definitions used by shared tests.

DOT aggregate

Dot is the aggregate root. Its primary relationships are:

```
Dot
 ├─── Skills[]                human-readable capabilities
 ├─── Contracts[]             authoritative interaction definitions
 ├─── Anchors[]               stable logical composition points
 └─── Runtimes[]
     ├─── Platform            where execution occurs
     └─── Deployments[]
         ├─── DeploymentManifest   how deployment occurs
         └─── DeploymentService[]
             ├─── Contract references
             ├─── ServicePort[]
             ├─── AnchorBinding[]
             └─── DeploymentConfig
```

`Skill` and `Contract` are independently identified domain entities referenced by ID. Anchors, Runtimes, Deployments, services, bindings, and configuration declarations are embedded values owned by the DOT aggregate.

The class-level `@ValidAnchor` constraint delegates aggregate traversal to `AnchorValidator`. It enforces Anchor identity, Contract membership, selector syntax, and service-port binding consistency without placing cross-reference logic in `Dot`.

Contract, Anchor, binding, and wire

These four concepts form a resolution chain but have different owners:

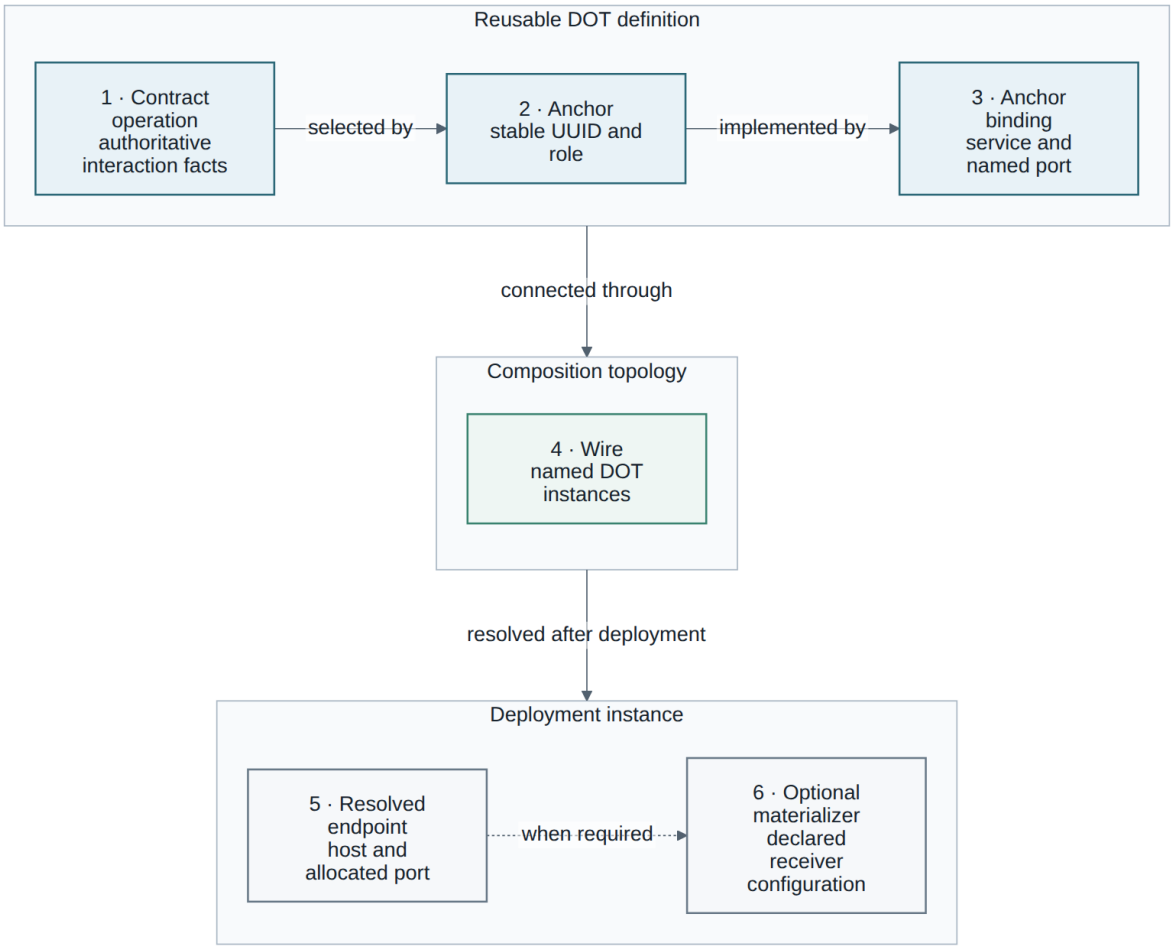


Figure 2. From an authoritative Contract operation to deployed configuration

- The **Contract** defines the interaction.
- The **Anchor** identifies one logical participation point.
- The **binding** says which declared service port implements that Anchor.
- The **wire** connects named deployment instances.
- The optional **materializer** writes a resolved value into configuration that the receiving service has already declared.

See DOT Anchors and Composition for the field model, compatibility rules, and examples.

Runtime and deployment separation

The model separates two orthogonal questions:

Concept	Field	Answers
`PlatformEngine`	`Runtime.platform.engine`	Where does the workload run? For example Docker, Kubernetes, Vagrant, or WebAssembly.
`DeploymentManifestType`	`Deployment.manifest.type`	How is it deployed? For example Docker image, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every manifest type declares its target platform. Runtime validation rejects a deployment whose manifest mechanism does not align with its platform.

`DeploymentService` is the unit that binds a deployed service to Contracts, named ports, Anchors, and supported configuration. Configuration is service-scoped; there is no deployment-level shared configuration surface.

Contract resolution

The reference resolver maps an Anchor selector to an authoritative Contract element:

```
AnchorContract(id, selector)
        |
        v
load Contract by UUID
        |
        v
standard-specific resolver
        |
        v
identifier + interaction + protocols + media types
          + input/output schema references + security scheme names
```

Resolution is read-only. Derived facts are returned to consumers and never written back into the Anchor. Contractless Anchors already carry their explicit direction, protocols, and media types and therefore have no selected Contract element.

Composition resolution

The provisional composition executor uses a compile-like pipeline:

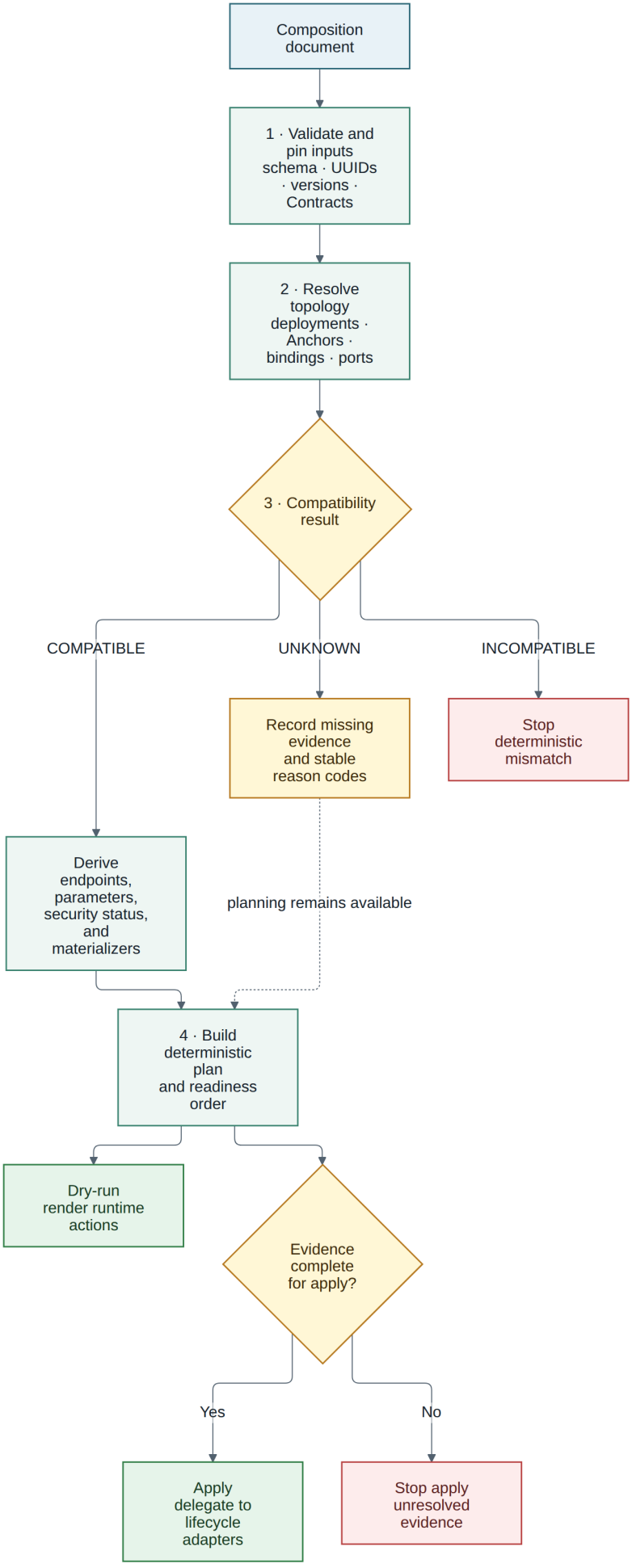


Figure 3. Fail-closed composition resolution pipeline

1. Validate the strict Draft 2020-12 composition shape.
2. Load each local DOT source and verify the pinned UUID and version.
3. Load each Contract input and verify its UUID.
4. Select one deployment for every named DOT instance.
5. Resolve Anchor references and service-port bindings.
6. Evaluate three-state compatibility: `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`.
7. Derive connection endpoints, parameter values, security status, and materializers.
8. Build a deterministic plan and readiness order.
9. On apply, prepare configuration and delegate workload operations to the existing single-DOT lifecycle adapters.

Planning may retain `UNKNOWN` with reason codes so missing evidence is visible. Apply rejects `UNKNOWN` rather than treating absence of evidence as compatibility.

Security boundaries

DOT-Design models declarations and explicit grants; it does not infer trust.

- Contract security requirements remain in the Contract.
- Sensitive deployment values are supplied by reference, not embedded in a composition or persisted as resolved values.
- OCI capabilities, host namespaces, volumes, and CDI devices require exact DOT declarations and exact operator grants.
- A configured wire does not prove credential validity, authorization, remote attestation, policy compliance, or network reachability.
- Cross-domain admission decisions belong to deployment or policy systems.

Active traffic and trust evidence must be collected by scenario-specific probes after deployment.

Catalogue and API boundaries

`DOT-Repository` versions reusable definitions and its catalogue metadata in Git. `dotx repository` validates and resolves those entries; `dotx compose` provides the executable compatibility reference. A future remote catalogue or API is free to use another language, framework, storage engine, and transport provided it preserves the domain semantics and passes the consumer acceptance profile.

`dotx compose` stores private, local lifecycle state under the user's cache directory. Per-composition locking prevents concurrent local lifecycle commands from racing that state. The current model is therefore explicitly single-host; shared orchestration state remains future work.

Design principles

- Reuse established Contract standards instead of inventing parallel protocol models.

- Keep stable logical identity separate from runtime location.
- Make adapters, transformations, privileges, and secret access explicit.
- Fail closed on ambiguity and missing security evidence.
- Preserve independently versioned DOT definitions; keep topology in composition.
- Treat generated plans and test evidence according to what they actually prove.

DOT Anchors and Composition

This guide explains how independently authored Dynamic Open Trust (DOT) components participate in composition by referring to authoritative Contracts rather than duplicating contract-defined facts. It distinguishes the stable DOT Anchor model from the provisional `dotx compose` reference format and follows the complete path from a logical capability to a deployed connection.

For a technology-neutral consumer boundary, see integration handoff.

Read this first



Keep these three boundaries intact:

1. A **Contract** describes what can be exchanged.
2. An **Anchor** gives one Contract operation a stable DOT-local identity and role.
3. A **wire** connects deployed DOT instances and remains outside both DOTs.

If a value changes between installations—for example a host, URL, port, credential, realm, broker, namespace, or device—it does not belong in an Anchor.

If you want to…	Start with
Publish a composable DOT	Authoring two composable DOTs, then Contract-backed Anchors and deployment binding.
Connect existing DOTs	Composition documents and DOT instances, direct request/response wires, and materializers.
Model messaging through a broker	Brokered wires and path and channel parameters.
Handle authentication or secrets	Security and secrets.
Operate a composition	Validation, planning, and lifecycle and troubleshooting.
Design a new 6G integration type	Anchor boundaries, extension points, and 6G security composition patterns.

Release status and supported profile

The DOT-owned Anchor model, contract selectors, service-port bindings, validation, generated schema, repository resolution, and conformance fixtures are release candidates. The separate composition document and its local lifecycle executor remain `dot.mare6g.eu/v1alpha1` reference functionality.

The current executable profile supports:

- local DOT sources pinned by UUID and version, and local Contract payloads pinned by UUID, in the composition;
- OpenAPI, AsyncAPI, Protobuf, and GraphQL contract selectors;

- contractless protocol/media declarations when no supported contract exists;
- named deployment selection and service-port bindings;
- explicit environment-variable and config-file materializers;
- Docker Image composition networking and per-instance identity;
- Kubernetes and Helm deployment/readiness through the existing single-DOT lifecycle;
- reference-only process-environment and Kubernetes Secret inputs;
- rollback, private local state, resumable partial apply, status, and reverse-order destroy; and
- fail-closed handling of ambiguity, unknown compatibility, unsupported privilege grants, and unreachable cross-platform materialization.

The authoritative machine-readable boundary is `DOT-Testing/acceptance/composition/v1alpha1/profile.json`. Run `task testing:compose:acceptance` to compare the installed CLI help and emitted schema with that profile, verify every committed lock, exercise all twelve cross-standard wires and six runtime dry-run adapters, and emit revision-pinned, secret-free JSON evidence.

It is not yet a general multi-host production orchestrator. Remote catalogue resolution, shared state, cross-platform network bindings, repeated-instance isolation for every deployment mechanism, external secret-store plugins, and mutating, multi-scheme, event-driven, or cross-platform probe adapters remain outside this profile. Safe anonymous OpenAPI probes and single-scheme OAuth 2.0 client-credentials probes within a shared Docker network or Kubernetes namespace are supported.

Single-DOT live lifecycle evidence for a runtime adapter is not automatically multi-DOT composition acceptance. The profile records validation, planning, dry-run, single-DOT live lifecycle, and composition live lifecycle separately for each adapter.

The five distinct objects

Object	Owner	Responsibility
Contract	DOT author	Authoritative operation, message, payload, protocol, and security description in OpenAPI, AsyncAPI, Protobuf, or GraphQL.
Anchor	DOT author	Stable UUID and composition role selecting one contract element, or declaring explicit communication facts when no contract exists.
Anchor binding	DOT author	Maps one Anchor UUID to a named port of a deployment service. It does not contain a live address.
Wire	Composer/operator	Connects one named DOT instance and Anchor UUID to another. It may select a provider, consumer, or broker service-port endpoint.

Object	Owner	Responsibility
Materializer	Composer/operator	Explicitly writes a resolved connection value into configuration declared by a participating service.

Where each fact lives

Fact	Source of truth	Referenced by
API operation, message, schemas, protocol, media type, security requirement	Contract payload	`Anchor.contract.selector`
Logical composition point and provider/consumer role	`Dot.anchors[]`	Wire `from` or `to`
Service and named port implementing an Anchor	`DeploymentService.anchorBindings[]`	Resolved plan endpoint
Configuration the service permits a deployer to supply	`DeploymentService.config`	Deployment input or materializer target
Selected DOT version, topology, readiness, parameter values, and secret references	Composition document	`dotx compose` plan and lifecycle
Concrete runtime address and port	Resolved deployment plan	Materializer or exposed-Anchor forwarding
Live reachability, authentication result, and policy decision	Runtime probe or external policy system	Operational evidence; never persisted as an Anchor fact

The ownership chain is:

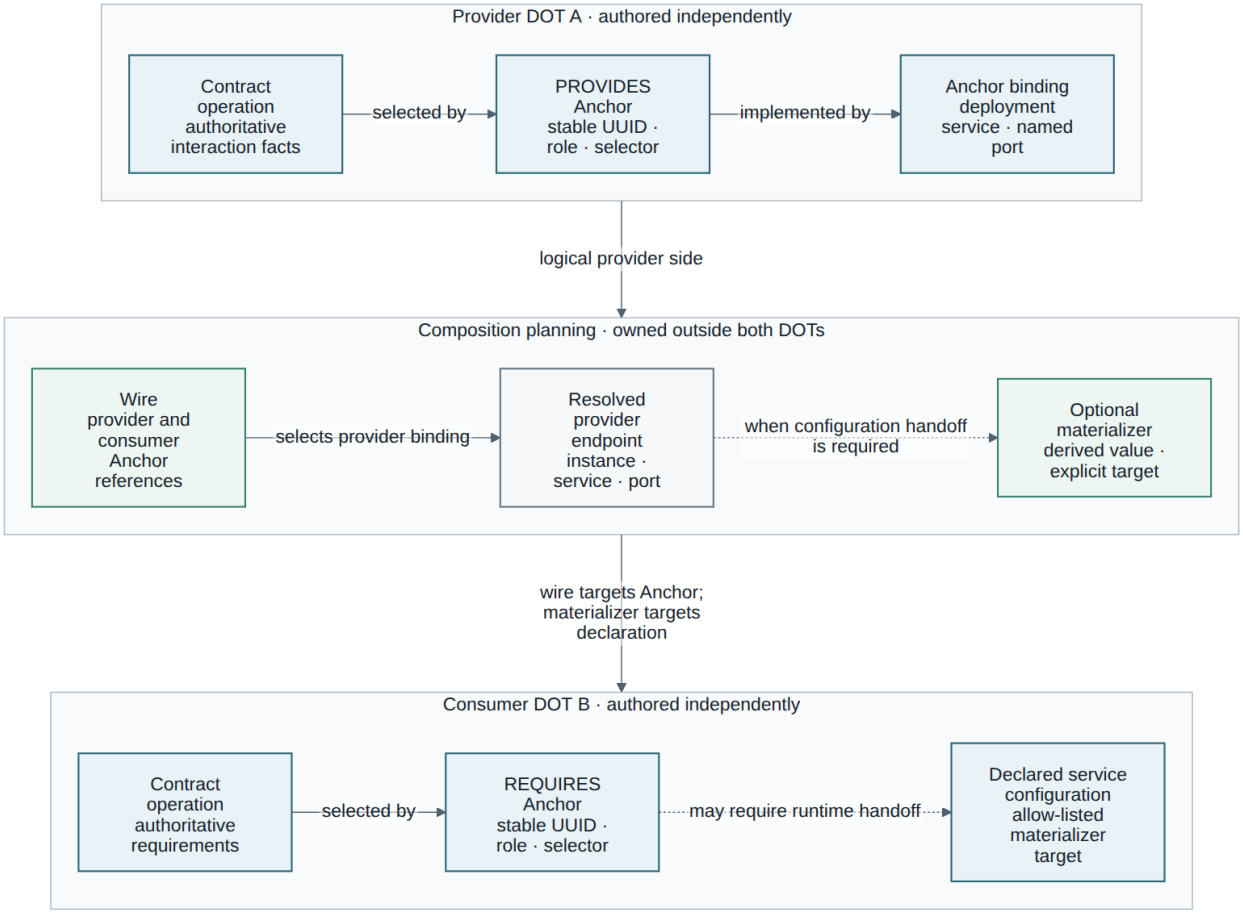


Figure 4. Provider-to-consumer ownership chain

These objects must not collapse into one schema. In particular, a wire is not part of either DOT, and a live URL is not part of an Anchor.

Resolution pipeline

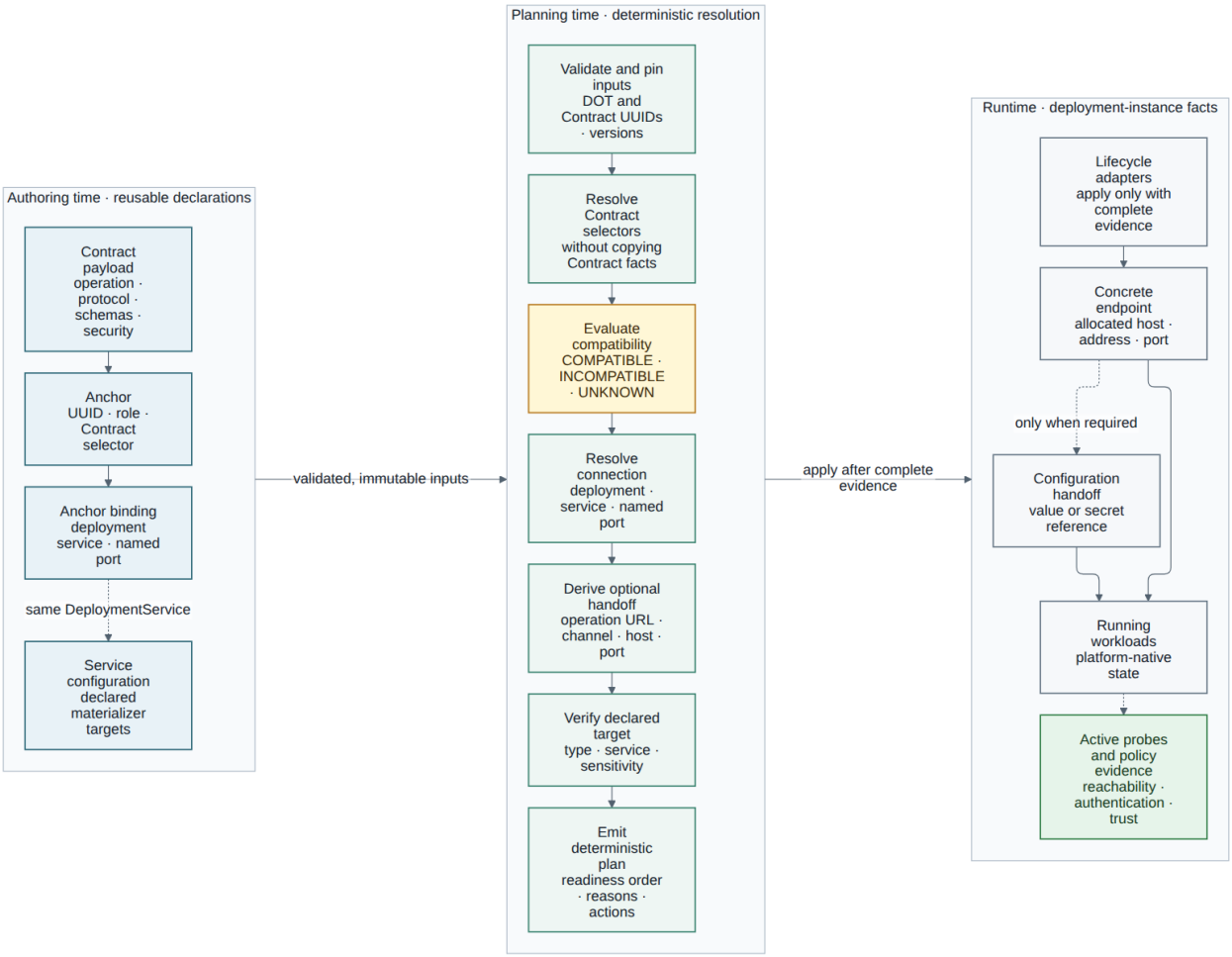


Figure 5. Authoring, planning, and runtime resolution

This resembles compilation: validate the independently authored inputs, link stable references, produce a deterministic deployment plan, and only then mutate runtime state. A failure at one phase is not evidence that a later phase was attempted.

Continue through the workflow

The detailed guide is organized around the work being performed:

Task	Guide
Publish stable composition points in a DOT	Authoring Anchors
Define pinned DOT instances and composition topology	Composition documents and instances
Connect endpoints, provide security inputs, and materialize configuration	Wiring, materialization, and security
Validate, plan, apply, inspect, and evolve a composition	Composition lifecycle and design guidance

Readers implementing a separate composer should also consult the integration handoff.

Authoring Anchors

This page covers the DOT publisher's side of composition. Start with the model overview if the distinction between Contracts, Anchors, bindings, and wires is not yet familiar.

Authoring two composable DOTs

Use this sequence when introducing a new connection:

1. Put the operation or message boundary in an OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract. Use the contractless form only when none of those standards can describe the boundary.
2. Add a stable `PROVIDES` Anchor to the provider DOT and a stable `REQUIRES` Anchor to the consumer DOT. Select the authoritative Contract element from each Anchor; do not copy its protocol or payload fields.
3. On every deployable implementation, bind each Anchor UUID to the named `ServicePort` that implements it.
4. Declare every consumer configuration input on its `DeploymentService`, including sensitivity. This declaration is the allow-list for composition.
5. Validate each DOT independently with `dotx validate` before writing a composition.
6. Add named, UUID/version-pinned DOT instances and UUID-pinned Contract inputs to a separate composition document.
7. Connect the Anchor UUIDs with a wire. Add an explicit broker endpoint, parameter values, security binding, or materializer only when the connection requires it.
8. Run `dotx compose lock` and commit the generated lock beside the composition. Review digest changes as changes to the resolved DOT-to-DOT graph.
9. Run `dotx compose validate` to check the document and `dotx compose plan` to inspect the resolved topology. Treat `UNKNOWN` compatibility as missing evidence, not as success.
10. Use `apply --dry-run` before execution. After apply, use `compose probe` for safe Contract-derived traffic and `status` for native runtime status, configuration evidence, and the latest redacted probe evidence. Dedicated acceptance tests remain necessary for authentication, authorization, mutable operations, and application-specific outcomes.

The complete three-DOT worked example is under `DOT-Testing/e2e/api-gateway-oidc/`: its `keycloak/`, `apisix/`, and `oauth-client/` directories contain independently valid DOTs; `contracts/` contains the authoritative APIs; and `scenario.yaml` owns the wiring.



Concrete identifiers in this guide use the canonical lowercase UUID text form with UUIDv4 version and IETF variant bits defined by RFC 9562. Deterministic values are retained where they identify executable repository fixtures, making the documentation traceable to the tested examples. Generate new UUIDs for production DOTs; do not reuse documentation or fixture identifiers.

Contract-backed Anchors

A contract-backed Anchor carries stable identity and selects one authoritative contract element. It deliberately omits protocol, direction, media type, payload schema, and security fields that the Contract already defines.

```
{
  "id": "a0000000-0000-4000-8000-000000000001",
  "name": "submit-alert-http",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:alert"],
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "selector": "#/paths/~1alerts/post"
  }
}
```

The opposite DOT declares its own `REQUIRES` Anchor. It may select the same Contract UUID or an independently authored Contract. Compatibility is deterministic when authoritative facts are comparable; otherwise the result is `UNKNOWN`, never an invented match.

Reading compatibility results

The plan compares the provider and consumer across independently explainable dimensions:

Dimension	Meaning
Role and direction	The connection is from `PROVIDES` to `REQUIRES`, and the selected interaction directions can communicate.
Semantics	Declared semantic-type URIs intersect when both sides provide them.
Protocol and media	Resolved protocol and media-type sets have a usable intersection.
Interaction	Request/response, publish/subscribe, streaming, query, mutation, or subscription behavior agrees.
Schema	Provider output and consumer input schema identities are compared in the correct direction when both select the same authoritative Contract; cross-Contract structural comparison remains `UNKNOWN`.
Security	Contract requirements are understood and, for execution, a complete supported binding has all required declared inputs.

`INCOMPATIBLE` means comparable evidence conflicts. `UNKNOWN` means evidence is missing or the current resolver cannot prove the dimension. Validate and plan may explain `UNKNOWN`; apply rejects it. An explicit mediator DOT is preferable to weakening compatibility when protocols or payloads genuinely differ.

Selector syntax

Standard	Selector	Example
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/paths/~1alerts/post`
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/operations/publishAlert`
Protobuf	Fully qualified service method	`mare.security.AlertService.PublishAlert`
GraphQL	Root-field schema coordinate	`Mutation.publishAlert`

For OpenAPI and AsyncAPI, `/` in a JSON Pointer token becomes `~1` and `~` becomes `~0`. Concrete path or channel template values belong to a composition wire's `parameters`, not to the Anchor.

Supported Contract representation matrix

Standard	Contract `format`	Authoritative interaction	Composition evidence
OpenAPI	`YAML` or `JSON`	Request/response operation, including path and security requirements	One shared selector; `REQUEST_RESPONSE` for both participants
AsyncAPI	`YAML` or `JSON`	Message operation and Channel Object	Distinct complementary selectors; `SEND` and `RECEIVE`, plus the resolved channel address
Protocol Buffers (Proto3; including gRPC service declarations)	`PROTOBUF`	Fully qualified RPC and unary or streaming shape	One shared method selector; `REQUEST_RESPONSE`, `CLIENT_STREAM`, `SERVER_STREAM`, or `BIDIRECTIONAL_STREAM`
GraphQL	`GRAPHQL`	SDL root field	One shared schema coordinate; `QUERY`, `MUTATION`, or `SUBSCRIPTION`

`dotx compose plan --json` exposes the resolved `contractStandard`, `contractFormat`, `providerInteraction`, and `consumerInteraction` on each contract-backed wire. These fields are derived from the supplied Contract and therefore do not duplicate authoring metadata in either Anchor or composition. The deterministic `DOT-Testing/composition/contract-standards/` matrix connects one provider DOT to one consumer DOT through every row above.

Semantic types

`semanticTypes` is a list of URI identifiers describing the security or domain meaning of the interaction independently of its transport. Prefer stable, governed ontology or vocabulary URIs when one exists; a project URN is suitable for concepts that have no external vocabulary yet.

Current matching is deliberately conservative: both Anchors must declare at least one semantic type and their sets must intersect exactly. Missing values produce `UNKNOWN`; different values produce `INCOMPATIBLE`. The evaluator does not infer subclass, equivalence, or broader/narrower ontology

relations. Such reasoning belongs in a separately reviewed semantic-policy layer.

Contractless Anchors

Use a contractless Anchor only when no supported authoritative contract exists, for example a raw packet stream or a legacy metrics boundary. It must state the facts that cannot otherwise be derived:

```
{
  "id": "a0000000-0000-4000-8000-000000000006",
  "name": "packet-stream",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:packet-stream"],
  "direction": "EGRESS",
  "protocols": ["TCP"],
  "mediaTypes": ["application/octet-stream"]
}
```

Contract-backed and contractless forms are mutually exclusive. A contract-backed Anchor must not repeat `direction`, `protocols`, or `mediaTypes`. A contractless Anchor must declare all three.

Role is not traffic direction

`role` answers who offers the selected capability:

- `PROVIDES` means this DOT makes the capability available;
- `REQUIRES` means this DOT depends on that capability.

It does not mean that bytes move in only one direction. An OpenAPI request/response operation has a providing server and requiring client while both send data. An AsyncAPI provider may select a `send` operation and its consumer a complementary `receive` operation. For a contract-backed Anchor, the resolver derives that interaction from the selected operation. Only a contractless Anchor states `INGRESS`, `EGRESS`, or `BIDIRECTIONAL` explicitly.

When two DOTs implement the same published Contract, referencing the same Contract UUID gives the strongest deterministic evidence. Independently authored Contracts are allowed, but identical-looking schema names do not prove structural compatibility across Contract identities; that comparison remains `UNKNOWN` until a schema-aware adapter or policy evaluates it.

Stable identity and version evolution

- The UUID, not `name`, identifies an Anchor in a wire.
- A rename or documentation correction preserves the UUID.
- A compatible DOT version preserves the UUID for the same logical Anchor.
- An incompatible semantic replacement receives a new UUID.
- Removal and any future `deprecated`/`replacedBy` metadata are publishing governance decisions; they are not inferred from contract changes.

- Names remain useful in logs and displays but are never foreign keys.

Binding an Anchor to a deployment

An Anchor is logical until a selected deployment maps it to a named service port:

```
{
  "id": "security-function",
  "name": "Security Function",
  "host": "security-function",
  "ports": [
    {"name": "http", "protocol": "HTTP", "port": 8080}
  ],
  "anchorBindings": [
    {
      "anchorId": "a0000000-0000-4000-8000-000000000001",
      "port": "http"
    }
  ],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": []
  }
}
```

This says which logical service and named port implement the Anchor. The deployer still decides the concrete container DNS name, cluster Service, allocated address, forwarded port, certificate, or network path.

`@ValidAnchor` and `AnchorValidator` reject duplicate Anchor UUIDs, undeclared Contract references, invalid selector syntax, undeclared ports, undeclared Anchor bindings, and duplicate service-local bindings.

Runtime configuration portability

Every `DeploymentService` declares its supported configuration surface through `environment`, `configFiles`, and logical `volumes`. A composition may supply only values whose names or target paths the selected service declares. Required environment variables and files without defaults fail before runtime mutation; sensitive environment values require a `secretRef` instead of a literal. When an input is omitted, `dotx` reuses the non-sensitive `defaultValue` or `defaultContent` declared by that service. Explicit composition values and wire materializers override defaults; conflicting explicit inputs fail closed. Declarations marked `sensitive` are forbidden from embedding either kind of default and must use a deployment-time secret reference or runtime-native secret facility.

Each composition input carries the stable `service` id it targets. Authors may omit `service` only when exactly one service in the selected deployment declares that environment name, file path, or volume name; planning resolves and records the service explicitly. If two services declare the same item, omission is an error. Duplicate detection is service-local, so two services can intentionally

receive different values for the same environment-variable name.

Environment variables and configuration files are rendered by every currently executable `dotx` deployment adapter:

Manifest	Environment	Configuration files
`DOCKER_IMAGE`	Container environment	Read-only bind mounts
`DOCKER_COMPOSE`	Per-service override using process-environment staging	Generated per-service Compose override mounts
`KUBERNETES_MANIFEST`	Exact workload/container patch; existing Secret keys remain `secretKeyRef`	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`HELM_CHART`	Exact post-install workload/container patch; existing Secret keys remain native	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`VAGRANTFILE`	Vagrant process/provisioner environment	Files provisioned into the managed VM and exposed to the Vagrantfile
`WASM_MODULE`	Allow-listed WASI guest environment; sensitive values inherited by name	Staged configuration root plus explicit Wasmtime directory preopens

Configuration remains service-scoped. A Docker image or Vagrant deployment is one workload or VM, so several logical services may safely share its declared configuration surface. Docker Compose, Kubernetes, and Helm may materialize several independent workloads. Their composition therefore supplies `deploymentInputs.workloads`: a Compose binding names one native service; a Kubernetes/Helm binding supplies exact native labels and optionally a container. Dotx intersects those labels with its composition-owned instance label and requires exactly one workload. Missing, duplicate, cross-runtime, ambiguous workload, and ambiguous container bindings fail closed. This prevents environment values, secrets, or mounted files from reaching the wrong security function without adding platform-specific selectors to the reusable DOT model.

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Storage and privilege configuration is deliberately runtime-native. Docker image deployments support composition-scoped named volumes, explicit OCI bind sources, exact capability/host-namespace grants, and CNCF CDI devices. Compose payloads own their volume graph; Kubernetes

and Helm own claims, storage classes, and volume topology; Vagrantfiles own disks and synced folders. A composer must not translate a host bind path into a cluster volume or silently grant privileged mode.

`WASM_MODULE` uses a reviewed Wasmtime/WASI profile. Each module is SHA-256 pinned and loaded from a relative/file or HTTPS URI. `WASI_CLI` executes command modules/components; `WASI_HTTP` serves a component on one declared TCP/HTTP service port bound to loopback. Dotx exposes only declared environment values, staged configuration files, and explicitly bound host directories. It does not translate OCI capabilities, host namespaces, or CDI devices into broader WASI permissions. Sensitive values are never printed or persisted. Because the Wasmtime CLI cannot enforce read-only permissions on a directory preopen, that request fails closed unless the host filesystem already supplies the boundary. The deterministic `DOT-Testing/composition/runtime-configuration/` matrix exercises every executable adapter. The `DOT-Testing/composition/service-workloads/` matrix verifies isolation between two services across Compose, Kubernetes, and Helm dry-run adapters.

Discovering and resolving Anchors

The authoritative test and reference definitions are catalogued in `DOT-Repository` and addressed as `repository://<dot-id>@<version>`. Consumers can inspect one immutable entry or validate the complete catalogue:

```
dotx repository inspect <dot-id>@<version> --repository DOT-Repository --json
dotx repository test --repository DOT-Repository --schema DOT-Schema/schema.json
```

`dotx compose validate` and `dotx compose plan --json` resolve Anchor UUIDs, service-port bindings, and selected Contract elements conservatively. The plan reports `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`; missing evidence is never treated as a match. Contract payloads remain authoritative and are not copied into Anchors or composition wires.

This command boundary is the reference behavior, not a requirement to embed dotx. Another catalogue or composer may use any language, framework, storage, or API shape if it preserves the same identities, fail-closed resolution, and ownership boundaries. See integration handoff.

Next, define the concrete participants and topology in a composition document.

Composition Documents and DOT Instances

A composition document is owned by the composer or operator, not by any member DOT. It selects immutable DOT identities and creates deployment instances whose topology may vary between environments.

Composition documents and DOT instances

A composition is a separate versioned document. It names concrete instances of DOT definitions and pins their identities through the authoritative `DOT-Repository` catalogue. The `repository://<dot-id>@<version>` form is stable across checkout locations and prevents a scenario-local copy from drifting away from the reviewed definition:

```
apiVersion: dot.mare6g.eu/v1alpha1
kind: Composition
id: e3000000-0000-4000-8000-000000000001
name: api-gateway-oidc
version: 1.0.0
dots:
  - name: identity
    source: repository://d3000000-0000-4000-8000-000000000001@1.0.0
    dotId: d3000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: kubernetes
  - name: gateway
    source: repository://d3a00000-0000-4000-8000-000000000002@1.0.0
    dotId: d3a00000-0000-4000-8000-000000000002
    dotVersion: 1.0.0
    deployment: kubernetes
    dependsOn: [identity]
contracts:
  - id: c3000000-0000-4000-8000-000000000001
    source: ./contracts/oidc-discovery.yaml
```

`name` identifies an instance inside this composition. `source` selects the catalogued definition by UUID and exact version. `dotx compose` locates the nearest `DOT-Repository/dot-repository.yaml`, resolves the entry to its reviewed bundle, and then verifies that `dotId` and `dotVersion` match the loaded `dot.json`. Neither a catalogue entry nor a local path can silently substitute a different DOT. `deployment` selects a deployment by its stable deployment ID or unambiguous manifest alias.

Relative paths remain supported for authoring experiments and external Contract documents. Reusable stable, sample, and test-only DOT definitions belong in `DOT-Repository`, under their corresponding catalogue scope. Contracts remain authoritative protocol documents and are referenced independently; placing a DOT in the repository does not copy OpenAPI, AsyncAPI, Protobuf, or GraphQL facts into its Anchors.

`dependsOn` is a readiness graph, not a connection graph. A wire does not imply startup order. This distinction permits reciprocal peer wires and brokered topologies without creating false dependency cycles.

Composition field reference

The emitted Draft 2020-12 JSON Schema is normative for the serialized `v1alpha1` shape. This table is the study guide for its top-level ownership:

Field	Required	Meaning
`apiVersion`, `kind`	Yes	Fixed discriminators: `dot.mare6g.eu/v1alpha1` and `Composition`.
`id`	Yes	Stable UUID of this composition, also used to derive local state and runtime identities.

Field	Required	Meaning
`name`, `version`	Yes	Human-readable identity and version of the composition document.
`dots[]`	Yes, at least one	Named deployment instances. Every entry pins `source`, `dotId`, and `dotVersion`; published scenarios use an exact `repository://<uuid>@<version>` source. `deployment`, `dependsOn`, and `deploymentInputs` refine execution.
`contracts[]`	When a referenced Contract is needed	Maps an exact Contract UUID to a local Contract document. The document's embedded `id` must match.
`wires[]`	No	Connections between one `PROVIDES` and one `REQUIRES` Anchor. A composition with no wires can still expose a standalone capability.
`exposedAnchors[]`	No	Composition-local names for member Anchors that an operator may expose or forward.

Each wire requires `name`, `from`, and `to`. Both ends contain an instance name and Anchor UUID. Optional `endpoint` selects a third-party or otherwise non-default service port; `parameters` supplies only Contract-declared template values; `materializer` or `materializers` writes resolved values; and `security` binds Contract security requirements to declared participant configuration. Unknown fields are rejected.

Deployment inputs and native workload bindings

`deploymentInputs.environment`, `configFiles`, and `volumes` identify the logical `DeploymentService` that owns each input. `ociRuntime` and `cdiDevices` authorize only requirements already declared by that selected DOT deployment. These fields supply instance values and grants; they do not expand the DOT's declared configuration surface.

When a Docker Compose, Kubernetes, or Helm deployment contains several logical services with configuration, `deploymentInputs.workloads` is required for every configured service. The binding is deliberately instance-owned because native service names, labels, and containers can differ between deployment variants or environments:

```
deploymentInputs:
  workloads:
    - service: api
      compose:
        service: backend
```

The Kubernetes/Helm form uses exact-match standard labels and may select one container:

```
deploymentInputs:
  workloads:
```

```
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Labels are combined with dotx's instance ownership selector. A live apply must resolve exactly one Deployment, StatefulSet, or DaemonSet, and an omitted container is accepted only when that workload has one container. Two logical services cannot target the same Compose service or the same Kubernetes workload/container pair. Plans expose the binding and secret identifiers, and locks cryptographically pin them; neither contains resolved secret values.

DOT definition versus DOT instance

One pinned DOT definition can appear more than once in a composition. Each entry has a distinct composition-local `name`, runtime identity, deployment inputs, and wires while retaining the same `dotId` and `dotVersion`:

```
dots:
  - name: site-a
    source: ./federation-agent
    dotId: d5000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-a}
  - name: site-b
    source: ./federation-agent
    dotId: d5000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-b}
```

Wires reference `site-a` or `site-b`, never the source path. Docker Image deployments receive deterministic per-instance container, DNS, network, and named-volume identities. Apply rejects repeated instances for deployment mechanisms whose manifest-owned resource names cannot yet be isolated safely.

The executable example is `DOT-Testing/composition/federation-peers/scenario.yaml`.

After defining the participants, wire and configure their selected Anchors.

Wiring, Materialization, and Security

Wires express composition topology outside every participating DOT. They refer to stable Anchor

UUIDs, resolve deployable endpoints through service bindings, and materialize only values that the target service has declared as configurable.

Direct request/response wire

A wire uses instance names and stable Anchor UUIDs:

```
wires:
  - name: oauth-token-to-client
    from:
      instance: identity
      anchorId: a3000000-0000-4000-8000-000000000002
    to:
      instance: client
      anchorId: a5000000-0000-4000-8000-000000000001
    materializer:
      type: ENVIRONMENT_VARIABLE
      valueFrom: OPERATION_URL
      target:
        instance: client
        service: oauth-client
        environmentVariable: TOKEN_URL
```

Planning performs the following sequence:

1. Resolve the two pinned DOT instances.
2. Resolve both Anchor UUIDs and require complementary `PROVIDES`/`REQUIRES` roles.
3. Resolve the selected Contract elements.
4. Evaluate semantic, interaction, protocol, media, schema, and security compatibility.
5. Resolve the connection endpoint through the selected deployment's `anchorBindings`.
6. Verify that the target service declares `TOKEN_URL`.
7. Build `OPERATION_URL` from the runtime endpoint and the authoritative OpenAPI server/operation path.

The resulting URL is deployment data. It is never copied back into `dot.json` or the Anchor.

A wire and a materializer are intentionally separate. The wire says *these Anchors are connected*; the materializer says *write this derived value into this declared consumer setting*. A wire needs no materializer when the runtime already provides discovery, a service mesh performs routing, both participants use a fixed broker declared elsewhere, or the composition is documenting an Anchor relation without an executable handoff. Apply still requires every actually required service configuration value to have a default or explicit input.

For the example above, the human-readable plan makes every resolution step inspectable before apply:

```
oauth-token-to-client:
  identity/a3000000-0000-4000-8000-000000000002
    -> client/a5000000-0000-4000-8000-000000000001 [COMPATIBLE]
  endpoint: keycloak:8080
    (HTTP, instance=identity service=keycloak port=http)
  contract: c3000000-0000-4000-8000-000000000002
    selector=#/paths/~1realms~1dotx~1protocol~1openid-connect~1token/post
  materializer: OPERATION_URL -> client/oauth-client env:TOKEN_URL
```

The same plan with `--json` is the machine-readable automation output. It includes stable instance and Anchor references, compatibility reasons, the selected Contract, resolved endpoint, derived operation/channel value, materializer targets, security status, and exposed-Anchor endpoints. Consumers should parse JSON rather than scrape the human-readable form.

Brokered wire

For messaging, the semantic provider, semantic consumer, and connection endpoint may be three different instances. Select the broker explicitly:

```
wires:
  - name: openc2-command-via-mqtt
    from:
      instance: interpreter
      anchorId: a6000000-0000-4000-8000-000000000003
    to:
      instance: actuator
      anchorId: a6000000-0000-4000-8000-000000000006
    endpoint:
      instance: broker
      service: mqtt
      port: mqtt
    materializers:
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_HOST
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_HOST
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_PORT
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_PORT
```

The AsyncAPI operations and message schema stay in the Contract. The broker's host and port stay in the resolved plan. If a channel has a templated address, the wire supplies only values for parameters declared by the Contract.

Materializers

Materializers are explicit adapters between a resolved endpoint and a configuration surface already declared by a service.

Type	Behavior
`ENVIRONMENT_VARIABLE`	Writes one resolved scalar to a declared environment variable.
`CONFIG_FILE`	Mounts a declared file and optionally replaces one YAML/JSON value selected by an RFC 6901 JSON Pointer.

Supported `valueFrom` values are:

Value	Result
`ENDPOINT_HOST`	Resolved service host only.
`ENDPOINT_PORT`	Resolved numeric service port only.
`ENDPOINT_HOST_PORT`	`host:port`.
`ENDPOINT_URL`	`scheme://host:port`, using the bound port protocol.
`OPERATION_URL`	Endpoint URL plus the authoritative OpenAPI server base and operation path.
`CHANNEL_ADDRESS`	Address of the authoritative AsyncAPI Channel Object.

An optional RFC 6570 `uriTemplate` can derive an application-specific URI from one resolved scalar. It is explicit composition behavior, not an Anchor or Contract fact.

Configuration-file materialization

A `CONFIG_FILE` materializer always names an operator-controlled source and a file path already declared by the target service. Without `valueFrom`, the source is supplied unchanged. With `valueFrom`, the source must be YAML or JSON and `jsonPointer` selects the scalar to replace:

```
materializer:
  type: CONFIG_FILE
  source: ./prometheus/prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

This produces a deployable Prometheus configuration without putting the provider's runtime address in either DOT. The corresponding executable fixture is `DOT-Testing/composition/poc4-continuous-monitoring/scenario.yaml`.

URI-template derivation

Use `uriTemplate` only when the application needs an explicit deterministic derivation that is not already the selected Contract operation URL:

```
materializer:
  type: ENVIRONMENT_VARIABLE
  valueFrom: ENDPOINT_URL
  uriTemplate: "{+value}/keycloak/realms/drax"
  target:
    instance: proxy
    service: proxy-xapp
    environmentVariable: OIDC_ISSUER
```

The RFC 6570 variable is named `value`. The template belongs to the composition because it adapts one deployment to another; it does not amend the Contract or redefine the Anchor.

Materialization fails when the target configuration is undeclared, the Contract path/channel remains unresolved, alternative server paths are ambiguous, two wires produce conflicting values for one target, or the endpoint and target are on different runtime platforms without an explicit network binding.

Path and channel parameters

Only deployment-specific scalar values belong in the wire:

```
parameters:
  realm:
    value: drax
```

The Contract remains authoritative for the parameter name, location, requirement, schema, default, and serialization rules. Current composition bindings cover OpenAPI path parameters and AsyncAPI channel parameters. Query/header/cookie parameters, complex serialization, and runtime-derived parameter values require explicit future adapters.

Security and secrets

Contract security remains authoritative. A wire may demonstrate how one declared scheme is satisfied by deployment configuration:

```
security:
  bindings:
    - scheme: bearerAuth
      mechanism: https://www.rfc-editor.org/rfc/rfc6749#section-4.4
      participant: {instance: client, service: oauth-client}
      parameters:
        tokenUrl: {environmentVariable: TOKEN_URL}
```

```
        clientId: {environmentVariable: CLIENT_ID}
        clientSecret: {environmentVariable: CLIENT_SECRET}
```

Sensitive inputs are references, never secret values. They are scoped to the DOT instance whose selected deployment consumes them:

```
dots:
  - name: client
    source: repository://d3a00000-0000-4000-8000-000000000003@1.0.0
    dotId: d3a00000-0000-4000-8000-000000000003
    dotVersion: 1.0.0
    deployment: kubernetes
    deploymentInputs:
      environment:
        - name: CLIENT_SECRET
          secretRef:
            store: kubernetes
            name: oauth-client
            key: client-secret
```

Plans and composition state keep only the reference. Docker deployments can resolve `store: environment` at final handoff; Kubernetes and Helm preserve `store: kubernetes` as `secretKeyRef` without reading the Secret. A `CONFIGURED` plan proves configuration completeness, not token validity, issuer trust, mTLS, workload identity, or remote attestation.

Exposed Anchors

A composition may re-export a member Anchor under a composition-local name:

```
exposedAnchors:
  - name: protected-metadata-through-gateway
    target:
      instance: gateway
      anchorId: a4000000-0000-4000-8000-000000000003
```

At apply time, an operator may request a local forward:

```
dotx compose apply scenario.yaml \
  --forward protected-metadata-through-gateway=19080
```

The exposure name belongs to the composition boundary. It does not rename or replace the member Anchor UUID.

Before runtime mutation, follow the validation and lifecycle workflow.

Composition Lifecycle and Design Guidance

The lifecycle deliberately separates reproducible input locking, model validation, deterministic planning, runtime mutation, recorded state, and active acceptance evidence. This keeps review and CI read-only while making execution recoverable and auditable.

Validation, planning, and lifecycle

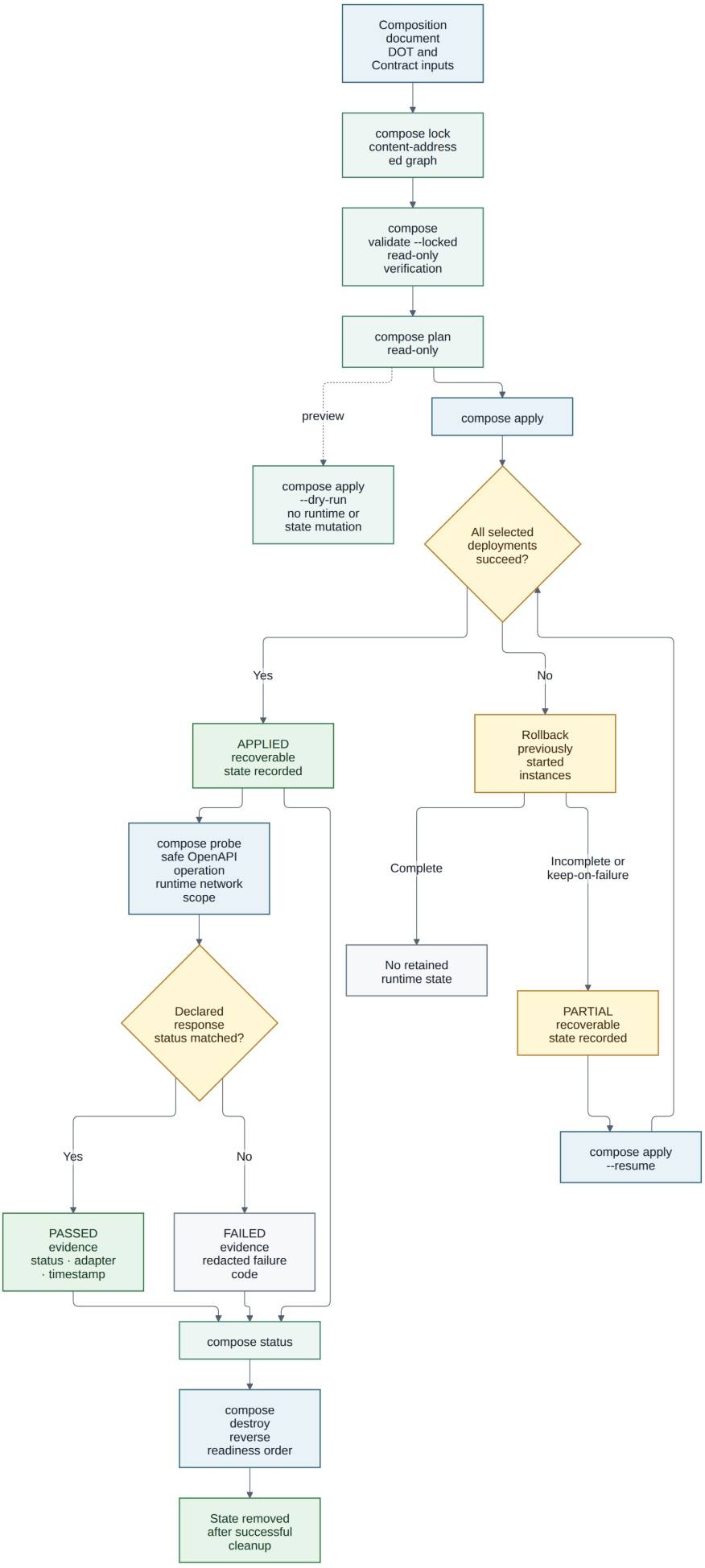


Figure 6. Composition lifecycle

What each command proves

Command	Proves	Does not prove	Mutates runtime/state
`compose lock`	The exact composition, selected DOT definitions and deployments, Contracts, configuration inputs, and portable resolved plan are content-addressed.	Runtime availability, credentials, or policy acceptance.	Writes only the requested lock file; `--check` is read-only
`compose validate`	The strict document shape, references, selected deployments, Anchors, Contracts, compatibility evidence, and materializer declarations resolve.	The exact runtime actions or live availability.	No
`compose plan`	Everything validated above plus the deterministic instance order, runtime identities, endpoints, Contract standard/format and participant interaction, derived values, security status, and materializers.	That the planned hosts are reachable or credentials work.	No
`compose apply --dry-run`	The composition reaches lifecycle preparation and can render supported deployment actions without persisting composition state.	Successful deployment or application-level traffic.	No
`compose apply`	The supported deployment adapters completed and recoverable state was recorded, subject to rollback behavior.	End-to-end Contract behavior unless separately probed.	Yes
`compose probe`	A selected safe OpenAPI operation is reachable from the composition's shared Docker network or Kubernetes namespace. Anonymous results match a declared response; supported OAuth results must match a declared `2xx` response.	Response-body conformance, mutating or multi-scheme operations, authorization-policy quality, AsyncAPI delivery, or full application semantics.	Creates and removes an ephemeral probe runner without changing member workloads; writes redacted probe evidence to composition state

Command	Proves	Does not prove	Mutates runtime/state
`compose status`	Native status for recorded instances, wire/exposure configuration evidence, and the latest recorded active-probe outcomes.	Current reachability after the probe timestamp, authentication mechanisms not explicitly identified by the evidence, authorization semantics, response bodies, or policy compliance.	No runtime mutation; reads state while holding the lifecycle lock
`compose destroy`	Owned instances were destroyed in reverse order and successful cleanup removed composition state.	Cleanup of resources not owned or recorded by this composition.	Yes

`validate` answers *is this composition internally resolvable?* `plan` answers *what exactly would be connected and materialized?* Keeping both commands read-only makes them suitable for review and CI.

`lock` makes those answers reproducible. Its JSON output preserves the original `repository://` and relative references while recording SHA-256 digests for the composition, every resolved `dot.json`, every supplied Contract, configuration inputs, and the portable resolved plan. It never records secret values, runtime addresses, or checkout-local absolute paths. Validate, plan, and apply verify an adjacent lock automatically; `--locked` additionally requires it, and `--lock <path>` selects an explicit file. Any mismatch fails before runtime mutation.

`probe` deliberately derives its request from the selected OpenAPI operation instead of introducing health-check fields in an Anchor. It accepts only `GET`, `HEAD`, and `OPTIONS`, requires one fully bound path, and executes a digest-pinned curl runner inside the common Docker network or Kubernetes namespace. The OAuth 2.0 adapter reuses one configured RFC 6749 client-credentials binding that satisfies a complete, single-scheme OpenAPI Security Requirement alternative. It resolves the provider-derived token URL, client ID, scopes, and client-secret reference at execution time, then requires a Contract-declared `2xx` response from the protected operation. Kubernetes Secret values remain `secretKeyRef` references; Docker secrets are supplied only through the runner environment. Neither client-secret values nor bearer token values appear in the runner argument vector or state. Contractless, mutating, multi-scheme, non-HTTP, unresolved, and cross-platform wires are reported as unsupported rather than inferred healthy. Transport diagnostics and response bodies are not persisted; state contains only the wire, Contract, operation, adapter, security-mechanism identifier, status, outcome, normalized failure code, and UTC timestamp.

The non-HTTP standards are fully covered for definition, selector resolution, compatibility, endpoint selection, materialization, locking, and deterministic planning. They are intentionally not treated as generically live-probeable: AsyncAPI needs broker/protocol delivery semantics and an observable receiver; gRPC needs a request encoded against the selected method and must account for streaming; GraphQL needs a valid operation document, variables, selection set, and inspection of the response `errors` member. Those values are interaction fixtures or application inputs, not Anchor fields. A protocol adapter must add that explicit evidence before claiming live acceptance.

```
# Emit the canonical strict Draft 2020-12 schema
dotx compose schema > composition-v1alpha1.schema.json

# Generate once, commit, and verify without rewriting
dotx compose lock scenario.yaml
dotx compose lock scenario.yaml --check

# Structure, references, selectors, compatibility, and materializer declarations
dotx compose validate scenario.yaml --locked

# Deterministic resolved topology; no runtime mutation
dotx compose plan scenario.yaml --locked
dotx compose plan scenario.yaml --locked --json

# Render runtime actions without state or runtime mutation
dotx compose apply scenario.yaml --locked --dry-run

# Execute, inspect recorded evidence, and clean up owned resources
dotx compose apply scenario.yaml --locked
dotx compose probe scenario.yaml --locked --wire oidc-provider-to-gateway
dotx compose probe scenario.yaml --locked --wire protected-gateway-to-client
dotx compose status scenario.yaml
dotx compose destroy scenario.yaml
```

Apply records each pending instance before runtime mutation, rolls back earlier instances on failure, and preserves recoverable state when rollback is incomplete or `--keep-on-failure` is selected. `apply --resume` requires an identical composition source, resolved plan, runtime options, inputs, ordering, and previously applied runtime state.

Apply, probe, status, and destroy acquire a private per-composition process lock. A concurrent lifecycle command fails immediately rather than racing local state or runtime mutations.

`compose status` reports:

- the recorded phase and native runtime status of each DOT instance;
- recorded wire compatibility, security configuration, selected endpoint, and materializer count; and
- recorded exposed-Anchor bindings; and
- the latest redacted active Contract-probe result for each probed wire.

Wire and exposure entries remain configuration evidence. Only entries under active Contract-probe evidence demonstrate a runtime request and declared OpenAPI response match, bounded by their timestamp and adapter scope.

Fail-closed rules

Apply rejects:

- `UNKNOWN` wire compatibility;
- missing, duplicate, or mismatched IDs and versions;
- ambiguous deployments, bindings, services, ports, or OpenAPI server paths;
- unresolved OpenAPI paths or AsyncAPI channel templates;
- undeclared materializer targets and conflicting target values;
- readiness dependency cycles;
- unsupported secret-store/runtime combinations;
- ungranted OCI capabilities, host namespaces, volumes, or CDI devices;
- repeated DOT instances on deployment mechanisms that cannot isolate their resource names; and
- materialized cross-platform wires without an explicit network binding.

Planning may retain `UNKNOWN` to explain incomplete evidence. Execution never silently upgrades `UNKNOWN` to compatible.

What does not belong in an Anchor

Do not add these as optional Anchor fields:

- hosts, URLs, numeric ports, broker addresses, or Kubernetes names;
- credentials, tokens, certificates, or secret values;
- Camel/Kamelet route syntax;
- transformation logic;
- placement, slice, tenant, trust-zone, latency, or availability policy;
- current health or live authorization state;
- host paths or operator device names; or
- orchestration dependencies.

Use Contracts for interaction semantics, service bindings for deployable ports, composition inputs for operator-controlled configuration, wires for topology, materializers or mediator DOTs for explicit adaptation, and policy engines for deployment/security decisions.

Choosing an extension point

Requirement	Preferred representation
New API/message operation	Add or update the authoritative Contract and select it from an Anchor.
Consumer needs a provider address	Wire materializer targeting declared service configuration.
Payload/protocol transformation	Explicit mediator DOT with two Contracts and two Anchors.

Requirement	Preferred representation
Startup/readiness ordering	Composition `dependsOn`.
Secret or certificate	Reference-only deployment input and platform/external secret provider.
Privileged runtime resource	Typed deployment requirement plus exact composition grant.
Broker service	Explicit wire `endpoint` selecting the broker service port.
Network attachment, eBPF/XDP, WASM import/export, or lifecycle event	Separately reviewed polymorphic binding type when a real use case requires it; do not grow `Anchor` with nullable fields.

6G security composition patterns

The same model supports security-function composition without baking one orchestrator's topology language into every DOT:

- **Zero-trust API chain:** identity-provider and policy-decision DOTs expose token, discovery, and authorization Anchors; an API gateway consumes them and exposes a protected application Anchor. Contracts carry OAuth/OpenID Connect and application semantics, while the composition supplies client-specific secret references and issuer URLs.
- **Closed-loop detection and response:** telemetry, inference, CACAO interpretation, and OpenC2 actuation remain separate DOTs. HTTP and MQTT Contracts define the boundaries; the wire selects the broker endpoint and supplies only deployment-specific channel values.
- **Multi-domain federation:** peer DOT instances can have reciprocal wires while `dependsOn` contains only real readiness dependencies. Trust-domain, roaming, and federation policy belongs in policy inputs or a dedicated policy DOT, not in the Anchor.
- **Near-real-time RAN control:** E2 or management APIs are Contract-backed Anchors. Host networking, accelerators, eBPF/XDP attachments, and radio-device access are explicit deployment requirements and grants, never inferred from a wire.
- **Observability and digital twins:** metrics, logs, events, and integrity APIs are distinct Anchors. A storage or message broker is selected explicitly as a connection endpoint; a shared host path is not treated as a portable integration boundary.

For cross-domain 6G deployments, a successful plan proves model consistency, not trust. Admission policy must still evaluate publisher identity, artifact provenance, attestation, slice/tenant placement, data-sovereignty constraints, and least-privilege grants before execution. Runtime acceptance should then probe the selected Contract operation over the actual network path.

Troubleshooting

Symptom	What to check
Anchor UUID is not found	The wire references the pinned DOT version, the UUID is stable, and the Anchor is present in that version's `dot.json`.

Symptom	What to check
Contract or selector cannot be resolved	The composition maps the exact Contract UUID to a readable source, the source document's `id` matches it, and the selector uses the standard-specific syntax.
Anchor has no deployable endpoint	The selected deployment contains exactly one service binding for the Anchor, and its named port exists on that same service.
Compatibility is `UNKNOWN`	Inspect `compatibilityReasons` in `compose plan --json`; add authoritative Contract evidence or use an explicit mediator. Do not override the result.
Materializer target is rejected	The target service declares the environment variable or config-file path, the JSON Pointer is valid, and no other wire writes a conflicting value.
Security remains unresolved	Bind a Contract-required supported scheme, declare all participant variables, and supply sensitive values by `secretRef` rather than plaintext.
Apply reports the composition is busy	Another apply, status, or destroy holds the per-composition lock. Let that command finish; do not delete the lock while a lifecycle process is running.
Status shows a configured wire but no active evidence	Run `dotx compose probe scenario.yaml --wire <wire>`. If the wire is skipped, its operation requires a dedicated adapter or application-level acceptance test. Configuration evidence alone is not reachability evidence.
An OAuth probe reports `AUTHENTICATION_ERROR`	Check the bound token URL and client ID, the referenced Secret name/key or environment-store source, required scopes, issuer reachability, and client authorization. Dotx deliberately does not persist the token endpoint's body.

Executable examples

- `DOT-Testing/conformance/anchors/` covers all four Contract standards, contractless Anchors, validation failures, bindings, and compatibility.
- `DOT-Testing/e2e/api-gateway-oidc/` is the canonical authenticated request/response lifecycle example.
- `DOT-Testing/composition/poc6-closed-loop/` demonstrates an explicit MQTT broker endpoint and OpenC2/AsyncAPI operations.
- `DOT-Testing/composition/federation-peers/` demonstrates reciprocal wires without false readiness cycles.
- `DOT-Testing/e2e/poc1-ric-aaa/` demonstrates identity, OPA authorization, forwarding, metrics feedback, and deny-list update across seven wires.
- `DOT-Testing/interoperability/pocs.yaml` is the evidence ledger for all eleven MARE PoCs and records every remaining acceptance gap.

Run the repository gate with:

```
task testing:conformance
```

The source-adapted E2E tests prove only their documented boundaries. They do not claim partner-runtime, live-RIC, production-network, trust-policy, or SFC acceptance unless that exact behavior is asserted by the scenario.

Verification and examples

DOT-Design keeps model correctness, resolved planning, deployment dry-runs, and live behavioral acceptance as separate evidence levels. A passing lower level never implies a higher one.

Evidence model

Level	Demonstrates	Does not demonstrate
Model and schema conformance	`dot.json` shape, Bean Validation constraints, stable references, selectors, service-port bindings, and generated-model compatibility	Contract payload resolution or multi-DOT topology
Contract resolution and compatibility	Selected operations exist and normalized semantic, interaction, protocol, media, schema, and security facts can be evaluated conservatively	Runtime reachability or valid credentials
Composition validation and plan	Pinned DOTs, wires, endpoints, materializers, parameters, grants, and readiness ordering resolve deterministically	Successful deployment
Lifecycle dry-run	The reference executor accepts the supported profile and renders runtime actions without writing composition state	Workload startup or traffic
Source-adapted E2E	Pinned upstream logic or artifacts execute behind explicit test adapters and satisfy the assertions documented by that scenario	Unmodified partner runtime, production infrastructure, or full PoC acceptance
Live production acceptance	The intended workloads, network, identity, policy, and Contract operations work in the target environment	Nothing beyond the probes and acceptance criteria actually executed

Primary verification commands

Command	Scope	External requirements
`mvn -pl DOT-Domain test`	Domain value objects, manifests, Anchors, and aggregate validation	Java and Maven only
`go -C DOT-CLI test ./…`	CLI parsing, validation, planning, deployment adapters, and lifecycle behavior	Go; individual tests use doubles where appropriate

Command	Scope	External requirements
`task testing:dots:repository`	Every definition admitted by `DOT-Repository/dot-repository.yaml`: catalogue identity, scope/path isolation, schema validity, CLI cross-references, and uniqueness	Go and the checked-out `DOT-CLI`, `DOT-Repository`, and `DOT-Schema` revisions
`task testing:compose:contract-standards`	Two real DOT definitions connected through OpenAPI 3.0/3.1/3.2 and AsyncAPI 3.0/3.1 YAML/JSON, Protocol Buffers (including gRPC service declarations), and GraphQL SDL; asserts selectors, participant interactions, endpoints, and materializers	`dotx`, `jq`, and the checked-out repository/schema revisions
`task testing:compose:acceptance`	Authoritative `v1alpha1` command/schema parity, every committed lock, twelve Contract wires, contractless compatibility, all six runtime dry-runs, service-scoped configuration, recovery/status/destroy, and focused fail-closed controls; emits revision-pinned JSON evidence	A globally installed `dotx` from the checked-out CLI revision, Go, Task, and fixture-specific dry-run CLIs
`task testing:contracts:standards`	Extracts the same twelve payloads and validates them with dated official OAI schemas, the official AsyncAPI CLI, `protoc`, and GraphQL.js; each family also has an invalid control which must be rejected	Python with PyYAML/jsonschema, Node.js, network access for pinned validators, and `protoc`
`task testing:compose:runtime-configuration`	The same catalogued DOT receives declared environment and configuration-file inputs through Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and Wasmtime/WASI dry-run adapters	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:compose:service-workloads`	A two-service DOT receives isolated defaults, overrides, configuration files, and secret references through exact Compose, Kubernetes, and Helm workload bindings	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:cli:coverage`	Complete DOT-CLI test suite plus machine-readable total and high-value package statement-coverage profiles	Go; writes reports below `DOT-Testing/e2e/target/`
`task testing:runtime:kubernetes-helm:live`	Real Kubernetes-manifest and Helm-chart deploy, rollout readiness, status, service-scoped environment materialization, and destroy lifecycle	Docker, Minikube, Kubernetes, Helm, and network access for runtime images

Command	Scope	External requirements
`bash DOT-Testing/runtime/wasm/live.sh`	Real Wasmtime lifecycle for WASI CLI; CI additionally builds the pinned Bytecode Alliance `wasi:http/proxy` sample and proves an HTTP response	`dotx`, Wasmtime, and the optional prebuilt HTTP component path
`task testing:conformance`	Cross-language Anchor and composition gate, including all deterministic PoC fixtures and lifecycle dry-runs	Java, Maven, Go, Task, and fixture-specific local tools
`mvn clean install`	Full Java domain build, schema generation, coverage gates, and SBOMs	Java and Maven only
`task testing:e2e:<scenario>`	Scenario-specific live behavior	Read that scenario's README first; commonly Docker and network access

Test asset layout

```
DOT-Testing/
 ├──── acceptance/        versioned, implementation-neutral consumer profiles
 ├──── conformance/       deterministic valid and invalid DOT fixtures
 ├──── composition/       read-only validate, plan, and dry-run scenarios
 ├──── coverage/          executable code-coverage ratchets
 ├──── e2e/               opt-in deployable behavioral scenarios
 ├──── interoperability/  revision-pinned evidence and remaining-gap ledger
 └──── runtime/           isolated native-runtime lifecycle harnesses
```

`testing` is repository-level infrastructure, not a Maven module and not a deployable DOT. Authoritative test-only definitions live under `DOT-Repository/DOTs/testing/` and are selected from compositions with `repository://<dot-id>@<version>`. Intentionally invalid conformance inputs stay under `DOT-Testing/conformance/`; they must never be admitted to the repository catalogue. Generated secrets, logs, reports, coverage profiles, and materialized configuration belong under scenario-local `target/` directories and must not be committed.

The focused `task testing:anchors:acceptance` gate verifies the versioned DOT Anchor consumer profile. It resolves pinned test definitions from `DOT-Repository`, verifies the composition lock, and asserts the complete cross-standard plan without a deployed workload.

The separate `task testing:compose:acceptance` gate freezes the complete `dot.mare6g.eu/v1alpha1` production profile. Its machine-readable source is `DOT-Testing/acceptance/composition/v1alpha1/profile.json`; the gate binds that source to CLI help, the tracked schema, repository locks, positive evidence, and negative fail-closed behavior. Runtime rows distinguish underlying single-DOT live lifecycle proof from multi-DOT composition lifecycle proof.

The focused `task testing:compose:contract-standards` gate complements that API profile with an executable composition matrix. It proves all six supported standard/format representations

between the same provider and consumer DOTs; it does not promote planning evidence to live protocol or application acceptance.

The separate `task testing:contracts:standards` gate validates those payloads outside the DOT resolvers with standards-native tooling. The positive fixtures must pass and deliberately malformed OpenAPI, AsyncAPI, Protobuf, and GraphQL controls must fail. This catches both invalid examples and accidentally permissive validator invocations.

The runtime-configuration gate separately proves that selecting another executable deployment does not discard a service's declared environment or configuration-file surface. Runtime-native storage, secret, privilege, and device semantics retain dedicated tests rather than being flattened into a misleading universal volume abstraction.

Coverage claims and gates

Coverage is reported by evidence dimension; the project does not collapse different meanings of "covered" into one percentage:

Dimension	Gate	Interpretation
Repository inventory	`dotx repository test`, `dotx repository audit`, and the DOT repository conformance workflow	Must be 100%: every catalogue entry is discovered, parsed, schema-valid, cross-reference-valid, uniquely identified, and stored in its declared scope; no deployable copy may remain outside `DOT-Repository`.
CLI statements	`task testing:cli:coverage`	Measured and published as an artifact. The whole CLI has a 35% baseline, with independent repository (80%), composition (75%), inspection (70%), Vagrant (70%), and WebAssembly (60%) package ratchets. This is a code-execution metric, not proof that a workload or security control works.
Java instructions and branches	`mvn verify` through JaCoCo 0.8.15	The Java domain module must retain at least 95% instruction and 78% branch coverage. JaCoCo HTML/XML/CSV reports are uploaded with the Java build evidence.
Native runtime lifecycle	WebAssembly and Kubernetes/Helm runtime E2E workflows	Proves real Wasmtime CLI and HTTP behavior plus Minikube manifest/Helm lifecycle. It does not imply that every external DOT workload is accepted.

Dimension	Gate	Interpretation
Composition scenarios	`task testing:e2e:all:dry-run`	Every listed deterministic scenario must validate, plan, and complete its supported lifecycle dry-run.
PoC interoperability	`DOT-Testing/interoperability/pocs.yaml` plus scenario-specific live workflows	Each of the eleven PoCs has traceable evidence, while remaining adapters, unavailable partner components, and unproved production behavior stay explicit.

The repository test gate fails on partial inventory, duplicate identities, mis-scoped definitions, or invalid DOTs. The workspace audit separately classifies uncatalogued, identical, divergent, and malformed definitions and fails on every deployable `dot.json` outside `DOT-Repository`. `DOT-Testing/conformance/` fixtures are reported but exempt because invalid definitions are their purpose. During the staged migration, run:

```
dotx repository audit --repository DOT-Repository --workspace . --json
```

The statement report deliberately has no artificial 100% threshold: a high line percentage cannot replace negative tests, lifecycle checks, live probes, or partner acceptance.

Canonical Anchor scenario

`DOT-Testing/e2e/api-gateway-oidc/` is the primary tutorial and lifecycle fixture. Its composition resolves three independently valid, test-scoped definitions from `DOT-Repository`:

- Keycloak provides OpenID Connect discovery and token operations;
- APISIX requires discovery information and provides a protected operation; and
- an OAuth client requires the token and protected operations.

The composition uses three OpenAPI Contracts, three wires, explicit security configuration, Kubernetes Secret references, configuration-file and environment materializers, readiness dependencies, and one exposed Anchor.

```
task testing:compose:api-gateway-oidc
task testing:e2e:anchors:api-gateway-oidc:dry-run

# Opt-in live execution
task testing:e2e:anchors:api-gateway-oidc
```

Read DOT Anchors and Composition alongside `DOT-Testing/e2e/api-gateway-oidc/scenario.yaml`.

MARE PoC-derived coverage

The interoperability ledger pins the source revision used for every analysis, records the modeled interactions, identifies the executable fixtures, and keeps remaining acceptance gaps explicit. The current repository contains source-adapted evidence for all eleven PoCs; this is not a claim of unmodified partner-runtime acceptance.

PoC	Composition focus	Fixture
1	OIDC identity, OPA authorization, RIC forwarding, metrics feedback, and deny-list update	`DOT-Testing/composition/poc1-ric-aaa/`
2	Repeated Flower trainers over gRPC/Protobuf and exposed BentoML inference	`DOT-Testing/composition/poc2-federated-inference/`
3	Five-stage contractless JSON insider-threat analytics dataflow	`DOT-Testing/composition/poc3-insider-threat-pipeline/`
4	Prometheus metrics, Elasticsearch logs, Grafana, and Kibana configuration	`DOT-Testing/composition/poc4-continuous-monitoring/`
5	Monitoring-to-analysis-to-assessment-to-mitigation security pipeline	`DOT-Testing/composition/poc5-security-pipeline/`
6	OpenAPI plus brokered AsyncAPI/MQTT, CACAO, OpenC2, and NWDAF remediation	`DOT-Testing/composition/poc6-closed-loop/`
7	Least-privilege packet capture, explicit CloudEvents mediation, and trust scoring	`DOT-Testing/composition/poc7-trust-scoring/`
8	cAdvisor/Prometheus observability and physical/twin integrity checks	`DOT-Testing/composition/poc8-ndt-observability/` and `poc8-twin-integrity/`
9	Amarisoft/Open5GS collection, InfluxDB storage, and data export	`DOT-Testing/composition/poc9-edge-ddos-collection/`
10	OIDC-protected capability exposure, SPARQL, OPA policy decisions, and observability	`DOT-Testing/composition/poc10-secure-capability-exposure/`
11	Reciprocal federation peers and local-core adapters with separate readiness edges	`DOT-Testing/composition/federation-peers/`

The authoritative coverage and gap list is `DOT-Testing/interoperability/pocs.yaml`. The official project descriptions are available from the MARE PoC catalogue.

Negative evidence is intentional

Rejected fixtures are part of the design evidence. They demonstrate that the model does not invent connectivity where the pinned source lacks a declared consumer input, does not treat stdout or a

host path as a portable connection, and does not silently grant privileged runtime access.

Examples include:

- PoC7 rejects a collector-to-scorer wire when the collector declares no such Anchor or configuration input;
- PoC8 rejects an undeclared Prometheus-to-NDT handoff; and
- invalid conformance fixtures cover missing Contracts, selectors, ports, duplicate bindings, and incompatible Anchor pairs.

Failing closed is a successful test outcome when the available evidence cannot support the proposed composition.

Adding or promoting a scenario

1. Pin the exact external source revision used as evidence.
2. Document what the source actually produces and consumes.
3. Add reusable deployable definitions to `DOT-Repository/DOTs/testing/`, record their evidence and purpose in `dot-repository.yaml`, and select them by exact `repository://<uuid>@<version>` identity.
4. Define Contracts only for supported, observable interactions.
5. Use contractless Anchors only when no supported Contract standard applies.
6. Declare all service ports, configuration targets, privileges, and devices.
7. Add negative fixtures for tempting but unsupported connections.
8. Start under `DOT-Testing/composition/` with validate and plan assertions.
9. Add a lifecycle dry-run before any live execution.
10. Promote to `DOT-Testing/e2e/` only with isolated resources, explicit prerequisites, reliable cleanup, and externally observable assertions.
11. Record what remains unproved in the interoperability ledger.

Interpreting results



A successful plan proves model consistency. A successful apply proves that the supported deployment adapters completed. Only an active scenario probe can prove that traffic, authentication, authorization, policy, or security behavior worked as intended.

Integration handoff

This handoff defines the interoperability boundary for a team implementing a catalogue, composer, Security Function Composer adapter, or other DOT consumer. It intentionally does not prescribe a service, database, API style, language, or framework.

Authoritative inputs

The reference source of reusable definitions is `DOT-Repository`. Resolve a DOT by its immutable UUID and pinned version, then verify the returned `dot.json` matches both. Resolve referenced Contract payloads without copying their protocol, schema, media, or security facts into an Anchor.

Use these commands as executable reference behavior:

```
dotx repository inspect <dot-id>@<version> --repository DOT-Repository --json
dotx repository test --repository DOT-Repository --schema DOT-Schema/schema.json
dotx compose validate composition.yaml
dotx compose plan composition.yaml --json
```

An implementation may expose equivalent behavior through REST, gRPC, events, an SDK, or an embedded library. Its external contract should be versioned and tested independently from the DOT model.

Required semantics

An interoperable consumer must:

- address DOTs, Contracts, and Anchors by stable UUID and pin DOT versions;
- resolve contract-backed selectors against the authoritative OpenAPI, AsyncAPI, Protocol Buffers, or GraphQL document;
- use `anchorBindings` to map a logical Anchor to a named service port;
- keep instance topology, endpoints, parameter values, credentials, and policy outside reusable DOT definitions;
- evaluate compatibility conservatively and preserve `UNKNOWN` as distinct from `COMPATIBLE`; and
- keep secret values out of DOTs, plans, persisted state, and logs.

Contractless Anchors are valid only when a supported Contract standard cannot describe the interaction. Their explicit direction, protocols, media types, and semantic types must be complementary before a wire is accepted.

Acceptance evidence

The versioned consumer profile is under `DOT-Testing/acceptance/dot-anchors/v1/`. It resolves two catalogued test DOTs, verifies the committed composition lock, and plans 12 compatible wires

across all supported Contract standards and versions.

```
task testing:anchors:acceptance
task testing:conformance
```

Passing the deterministic profile proves discovery and planning semantics. A joint live test must separately prove endpoint reachability, credential and token handling, authorization, policy enforcement, and application behavior.

Future implementation freedom

The removed catalogue-service prototype is not part of the production architecture. A future team may replace it or implement discovery elsewhere in a different language or framework. Compatibility is judged by the stable DOT model and executable acceptance profile, not by reproducing the prototype's internal classes or former HTTP response shapes.

DOT-Domain

Domain model

`DOT-Domain` is the persistence-neutral Java library that defines the portable DOT aggregate and its validation rules. It contains no application, web, or storage-adapter layer.

Aggregate overview

Type	Responsibility
`Dot`	Aggregate root: identity, version, description, classification groups, metadata, Skills, Contracts, Anchors, and Runtimes.
`Skill`	Standalone human-readable capability description.
`Contract`	Standalone authoritative OpenAPI, AsyncAPI, Protobuf, or GraphQL document.
`Anchor`	Embedded stable logical point through which the DOT participates in composition.
`Runtime`	A target `Platform` plus one or more compatible deployment definitions.
`Deployment`	Stable deployment ID, one polymorphic manifest, and the logical services it materializes.
`DeploymentService`	Service identity, host, named ports, Contract references, Anchor bindings, and its supported configuration surface.

`Dot`, `Skill`, and `Contract` are independently identified domain entities. Other types are embedded values owned by a DOT. The module carries no database or ORM annotations; persistence is an adapter concern.

Identity, versioning, and references

Root entity IDs are UUIDs. Equality for `Dot`, `Skill`, and `Contract` is ID-based so changes to mutable metadata or collections do not destabilize sets or persistence identity.

`Version` accepts Semantic Versioning 2.0.0 values and the special `latest` selector. It serializes as a string and implements SemVer precedence; `latest` sorts after concrete versions. A new `Dot` or `Contract` defaults to `0.0.0`, while `Platform.version` defaults to `latest` so no platform-specific release is implied.

JSON serializes `Dot.skills`, `Dot.contracts`, and `DeploymentService.contracts` as ID-only objects. Consumers resolve those UUIDs against authoritative definitions rather than relying on object embedding.

Catalogue uniqueness rules are:

- `Skill.name` is globally unique;

- `Dot` is unique by `(name, version)`; and
- `Contract` is unique by `(name, version)`.

Contracts

Standard	Payload format	Anchor selector
`OPENAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`ASYNCAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`PROTOBUF`	`PROTOBUF`	Fully qualified service method
`GRAPHQL`	`GRAPHQL`	Root-field schema coordinate

`Contract.isFormatCompatibleWithStandard()` rejects mismatched standard/format pairs. Payload parsing and selected-element existence are resolver concerns; the domain model validates selector syntax and references within the aggregate.

Anchors and bindings

A contract-backed Anchor contains:

- a stable UUID and display name;
- a `PROVIDES` or `REQUIRES` role;
- optional semantic-type URIs; and
- a Contract UUID plus standard-specific selector.

It omits direction, protocol, and media type because those facts come from the selected Contract operation. A contractless Anchor must declare `direction`, at least one protocol, and at least one media type explicitly.

`DeploymentService.anchorBindings` maps an Anchor UUID to a named `ServicePort` on that same service. An Anchor contains no host, numeric port, credential, placement, or live policy state.

The class-level `@ValidAnchor` constraint delegates cross-reference validation to `AnchorValidator`. It rejects:

- duplicate Anchor UUIDs within one DOT;
- Contract references not declared by the same DOT;
- selectors invalid for the referenced Contract standard;
- bindings to undeclared Anchors;
- bindings to port names absent from that service; and
- duplicate bindings of one Anchor within the same service.

See DOT Anchors and Composition for authoring and wiring examples.

Runtime two-layer model

`Runtime` separates infrastructure from deployment mechanism:

Enum	Used by	Answers
`PlatformEngine`	`Platform.engine`	Where the workload runs: `DOCKER`, `KUBERNETES`, `VAGRANT`, or `WEBASSEMBLY`.
`DeploymentManifestType`	`DeploymentManifest.type`	How it is deployed: Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every `DeploymentManifestType` declares its required `PlatformEngine`. `Runtime.isDeploymentPlatformAligned()` validates every deployment against the runtime platform. `Runtime.isDeploymentIdsUnique()` enforces stable deployment selection within that runtime.

`Deployment.services` is a set. `DeploymentService` uses ID-based equality, so logical service IDs remain unique within the deployment.

Deployment manifests

`DeploymentManifest` is polymorphic JSON dispatched by its existing `type` property.

Type	Java subtype	Important fields
`DOCKER_IMAGE`	`DockerImage`	Image reference and ordered dotenv payloads
`DOCKER_COMPOSE`	`DockerCompose`	Complete YAML/JSON payload and ordered dotenv payloads
`KUBERNETES_MANIFEST`	`KubernetesManifest`	Complete YAML/JSON manifest payload
`HELM_CHART`	`HelmChart`	Repository, chart or OCI reference, version, and ordered values payloads
`VAGRANTFILE`	`Vagrantfile`	Complete Ruby DSL payload
`WASM_MODULE`	`WebAssemblyModule`	Relative/file or HTTPS module URI, WASI execution world, command arguments, and required SHA-256 integrity pin

Docker Compose and Kubernetes manifest payloads accept YAML or JSON only. Helm values also preserve YAML/JSON order: later entries override earlier entries as with repeated Helm `-f` arguments.

Runtime instance concerns do not belong in a manifest. For example, a Helm release name and namespace are selected by deployment tooling, not embedded in `HelmChart`.

Service configuration declarations

Every `DeploymentService` owns one `DeploymentConfig` containing:

- environment variables, including default, required, and sensitive metadata;
- configuration-file paths, content type, defaults, and sensitivity;
- logical volume mounts;
- logical CNCF CDI device requirements; and
- optional OCI capability and host-namespace requirements.

These are supported inputs and requirements, not applied runtime state. Operator values, host paths, concrete CDI device names, and permission grants belong to deployment or composition input.

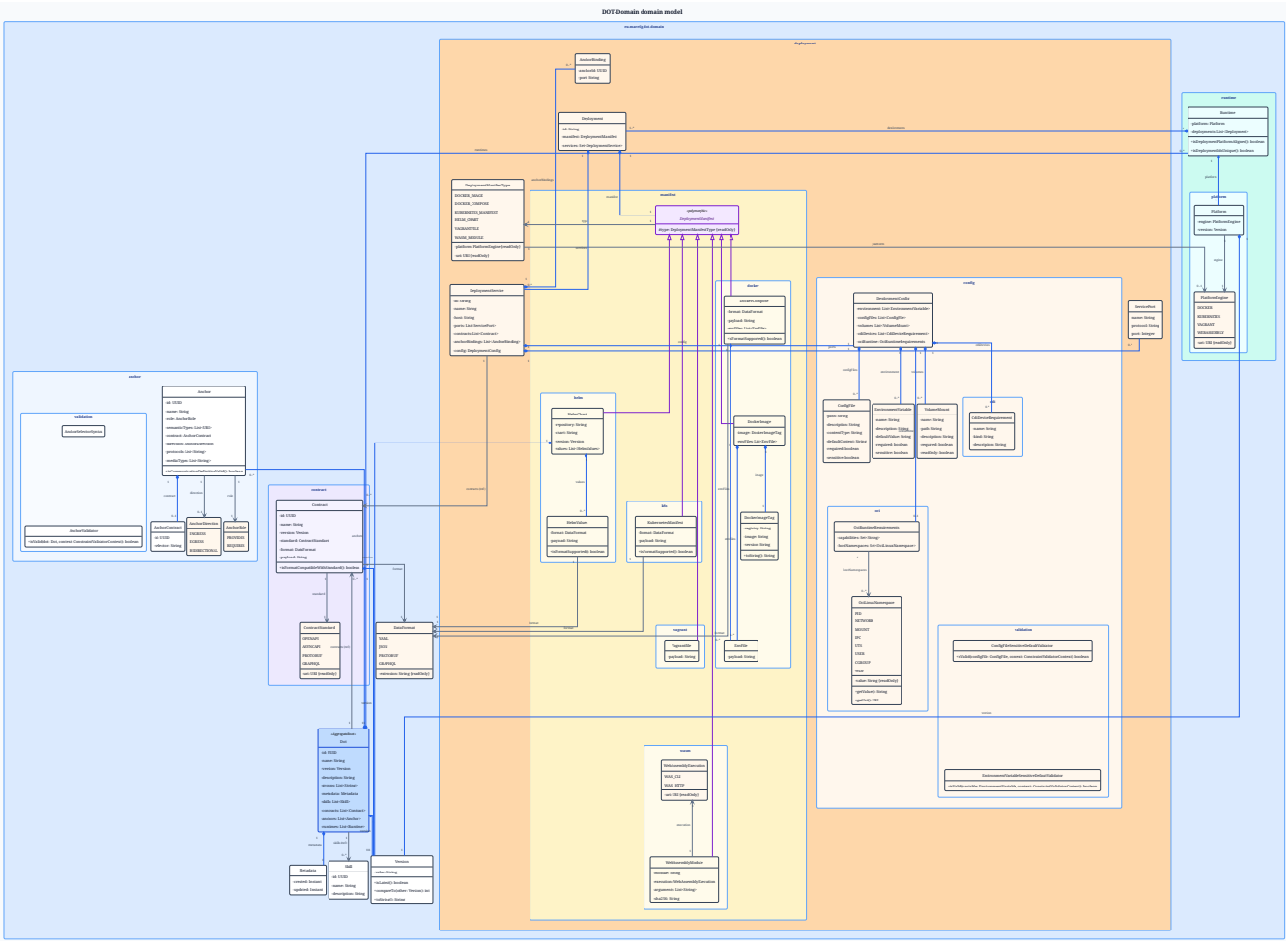
OCI requirements use standard `CAP_*` capability names and typed Linux namespaces. The model deliberately has no unrestricted `privileged` flag.

A CDI requirement declares a stable logical name and a `vendor.example/class` kind. A composition binds it to one exact `vendor.example/class=device` identifier. CDI installation, allocation, and authorization remain operator responsibilities.

Serialization and persistence adapters

- Collections have initialized builder defaults and do not become `null` when constructed through Lombok builders.
- `Version` is JSON-serialized as a normalized string.
- Persistence adapters must preserve UUID and version representations.
- Referenced `Skill` and `Contract` objects expose only `id` when nested in DOT JSON.
- Domain values carry no database or ORM annotations.

Class diagram



The diagram is generated from current domain sources during the DOT-Domain build. Open it separately for the full-resolution view.

Build and test

```
# Unit tests only; no service or database required
mvn -pl DOT-Domain test

# Regenerate schema and diagram through the reactor lifecycle
mvn -pl DOT-Domain clean install
```

Representative tests cover version precedence, reference serialization, manifest polymorphism, runtime/manifest alignment, deployment configuration, Anchor rules, and entity equality.

Related pages

- Architecture
- DOT Anchors and Composition
- Integration handoff